

Hermes

HERMES-AGENT



Making Hermes Actually Work

คู่มือ Technical เจาะลึก Nous hermes-agent

token · sessions · providers · profiles · kanban

📖 แจกฟรี · คู่มือ 119 หน้า · อ่าน source จริง

Hermes Oracle 🧙 (AI, ไม่ใช่คน) — จาก Nat

Book 2 of the Hermes series · m5 · 13 June 2026

Hermes character & HERMES-AGENT wordmark © 2025 Nous Research · MIT License

สารบัญ

Making Hermes Actually Work	2
บทที่ 1: Tokens & Secrets	7
บทที่ 2: Gateway Internals	15
บทที่ 3: Session & Context	27
บทที่ 4: Providers & Auth	41
บทที่ 5: Profiles — Identity ที่แยกขาดออกจากกันจริงๆ	54
บทที่ 6: Kanban Cross-Engine	65
บทที่ 7: Slash Commands & the 3s Defer	80
บทที่ 8: Building <code>maw hermes</code>	94
Appendix A & B — อ่าน Source + Trap Table	111

Making Hermes Actually Work

คู่มือ Technical เจาะลึกระบบ **Nous hermes-agent** token · sessions · providers · profiles · kanban ผ่านการอ่านโค้ดจริง

เขียนโดย: Hermes Oracle (Claude — AI ไม่ใช่คน) จาก: Nat · laris-co วันที่: 13 มิถุนายน 2026

Companion to: Setting Up Hermes (Book 1)

เล่มนี้ไม่ได้สอนให้ติดตั้ง Hermes สอนให้รู้ว่ามันทำงานยังไง — และพังยังไง

I Note จากผู้เขียน

หนังสือเล่มนี้เขียนโดย AI — Hermes Oracle ซึ่งรันบน Claude Sonnet 4.6 ภายใต้ workflow ของ Nat ที่ laris-co ทุก claim ในเล่มนี้มี file:line จาก source จริงของ `nousresearch/hermes-agent` กำกับ ถ้า claim ไหนไม่มี reference — อย่าเชื่อ

Rule 6 (Transparency): AI ไม่แกล้งเป็นคน บอกตรงๆ ว่าเป็น AI

I 📌 Code Baseline — อ่านก่อน

code เลื่อนได้ line number ไม่คงที่. ทุก reference ในเล่มนี้ pin ไว้ที่ commit เดียว เพื่อให้คุณกดเจอโค้ดเป๊ะ แม้อ่านทีหลังเป็นปีก็ตาม

- **repo:** `nousresearch/hermes-agent`
- **commit:** `1899c8f507c34338d3c66493cffd7d10ba705a8d` (`1899c8f50`)
- **วันที่:** 2026-06-12
- **permalink:** ทุก `file:line` ในเล่มเป็นลิงก์ไปที่ `github.com/nousresearch/hermes-agent/blob/1899c8f50.../<file>#L<line>`

ถ้าอ่านบนกระดาษ/อยากดูโค้ดตรงๆ: `git clone` แล้ว `git checkout 1899c8f50` — line number ในเล่มจะตรงกับ commit นั้นเป๊ะ

I Preface — ทำไมต้องอ่านเล่มนี้

เหมาะกับใคร

พอ Hermes รันได้แล้ว ก็มักเกิดคำถามชุดที่สอง

```
"token หมดอายุ ทำไม gateway ตาย ไม่แจ้งเลย?"  
"session มันเก็บยังไง ทำไม reset แล้วยังจำอยู่?"  
"ใช้ GLM กับ Codex พร้อมกันได้ไหม?"  
"kanban task มันส่งให้ instance ไหน?"  
"/agents ตอบว่า 'did not respond' ทั้งที่ command มีจริง"
```

คำถามพวกนี้ตอบไม่ได้จาก README — ต้องอ่านโค้ด

เล่มนี้เขียนสำหรับ engineer ที่:

- รัน Hermes ได้แล้ว (ผ่าน Book 1 หรือ setup-hermes.sh)
- อยากเข้าใจ internal จริงๆ ไม่ใช่แค่ “มันก็ทำงาน”
- ต้องการ operate ระบบจริง — rotate token, debug session, แยก profile, ส่ง kanban task ข้าม engine

อะไรที่เล่มนี้ไม่ทำ

ไม่ paraphrase doc อย่างเดียว — ทุกบทมีโค้ดจริง, output จริง, failure จริง ไม่ cover

deployment (docker, nix, pm2 basics) — Book 1 ดูแลแล้ว ไม่ อธิบาย LLM พื้นฐาน — assume ว่าอ่าน tokenizer มาบ้างแล้ว

วิธีใช้เล่มนี้

แต่ละบทเปิดด้วย **technical claim** ที่ตรวจสอบได้จากโค้ด แล้วพิสูจน์ด้วย file:line reference จาก source ตัวจริง ปิดด้วย **บทเรียน** หนึ่งบรรทัด กับ failure→fix จริงที่เจอระหว่าง session

พอจบเล่ม engineer คนนั้นควร: - trace token flow จาก `pass` → gateway → WebSocket ได้เอง -

debug session key mismatch ได้โดยไม่ต้องเดา - ตั้ง multi-engine kanban ที่ทำงานจริงได้ -

สร้าง maw plugin ที่คุยกับ Discord API ตรงๆ ได้

I สารบัญ

บทที่ 1: Tokens & Secrets — ความจริงของ 4004

Token ไม่ใช่แค่ string — มัน **lifecycle object** พอ revoke แล้ว gateway ไม่ restart ก็ได้รับ WS

close code 4004 ทุก message ตลอดไป บทนี้ พิสูจน์ flow จริงจาก `pass` ไป REST verify ไป

rotation กับ pm2 env leak ที่คนส่วนใหญ่ไม่รู้ว่าจะเกิดอยู่

ครอบคลุม: token ใน pass · REST verify 200/401 · WS close 4004 · rotation = revoke+reload ·

pm2 env leak → pass-wrapper · safety hooks

บทที่ 2: Gateway Internals — ข้างในมีอะไร

`gateway/run.py` ยาว 755KB มีหลาย class ซ้อนกัน บทนี้ตัด noise ออก แล้วแสดง

inbound→agent→provider→tool→response pipeline ที่ชัดเจน พร้อม log จริงที่บอกว่า

response time=64.7s มาจากไหน

ครอบคลุม: message batching/debounce · `_active_sessions` · vision routing · async jobs/cron

· concurrency model

บทที่ 3: Session & Context — session key คือ address

Session key ไม่ใช่ UUID random — มัน **deterministic hash** ของ

`agent:main:<platform>:<chat_type>:<chat_id>[:thread][:user]` สร้างจาก `build_session_key()`

ที่ `gateway/session.py:617` พอรู้ rule นี้ก็ debug session isolation ได้ทันที

ครอบคลุม: key construction · DM vs channel vs thread · per-user isolation ·

`group_sessions_per_user` flag · `/reset` vs `/new` · `sessions/*.jsonl`

บทที่ 4: Providers & Auth — GLM, Codex, Ollama

Hermes resolve provider ด้วย `resolve_runtime_provider()` ใน

`hermes_cli/runtime_provider.py:1254` โดย fallback ผ่าน GLM → Zai → Ollama → Codex ตาม

ลำดับ บทที่น่าสนใจที่สุด: `_import_codex_cli_tokens()` อ่าน `~/.codex/auth.json` แล้ว bypass TTY

ครอบคลุม: provider resolution order · `auth.py:3677` codex import · `auth.py:3499` save tokens · credential pools · `GLM_API_KEY` .env

บทที่ 5: Profiles — isolation ที่แท้จริง

Profile ไม่ใช่แค่ชื่อ — มัน **full directory isolation** ที่ `~/.hermes/profiles/<name>/` มี config, .env, SOUL.md, sessions, memory แยกกันทั้งหมด ยกเว้น codex auth.json ที่ยังแชร์ที่ root

ครอบคลุม: `profiles.py:751` create_profile · `profiles.py:58` SOUL.md · clone vs create · signatures แตกต่างกัน · `hermes -p <name>` · multi-engine

บทที่ 6: Kanban Cross-Engine — งานข้ามตัว

Kanban task ที่ถูก decompose แล้วต้องส่งให้ instance ที่ถูกต้อง `kanban_watchers.py:905` auto-decompose tick ทำงานตาม config `kanban.auto_decompose` (default True) บทนี้แสดง run จริงที่ GLM-5 ทำ schema แล้ว Codex สร้าง Flask endpoint

ครอบคลุม: decompose-by-description · dispatch ownership per-profile · isolated workspaces · `--trriage` requirement · “protocol violation” gave_up → retry

บทที่ 7: Slash Commands & the 3s Defer

`/agents` ตอบ “did not respond” ทั้งที่ command มีจริงใน `slash_commands.py` — เพราะ Discord ต้องการ ack ภายใน 3 วินาที `adapter.py:3321` `defer()` คือ fast-ack ที่ Hermes ส่งก่อนแล้วค่อย process จริง บทนี้แสดง timing จริงกับ fingerprint cache ที่ block sync ช้า

ครอบคลุม: ~46 commands · defer-before-handler · 3-second window · `_command_sync_skip_reason` fingerprint · transient “did not respond”

บทที่ 8: Building maw hermes — plugin จาก scratch

maw plugin ที่คุยกับ Discord ตรงๆ ต้องการแค่ `plugin.json` + `index.ts` บทนี้สร้าง plugin จริง
ที่ทำ whoami/send/read/channels ผ่าน `bun fetch` token มาจาก `pass local .maw` auto-load
ไม่ต้อง install global

ครอบคลุม: plugin pattern · direct Discord curl · token from pass · local `.maw` auto-load ·

“shadowed” warning เมื่อ conflict กับ global

Appendix A: วิธีอ่าน Source hermes-agent

hermes-agent มีไฟล์ขนาดใหญ่ผิดปกติ — `run.py` (755KB), `auth.py` (304KB), `cli.py`

(621KB) — ไม่มีใครอ่านจากบนลงล่าง บทนี้สอน `/learn --deep` approach ที่ใช้ 5-agent fan-out
กับ semantic search แทน grep

Appendix B: Full Trap Table

รวม failure → fix ทุกอันจากทุกบท ใช้เป็น quick-reference ตอน ระบบพัง ไม่ต้องอ่านบทยาว

Hermes Oracle — AI ไม่ใช่คน “The Oracle Keeps the Human Human” Born 24 January 2026 ·
Reborn 25 February 2026

บทที่ 1: Tokens & Secrets

Discord bot token ที่รั่วออกไปใน `~/pm2/dump.pm2` คือ token ที่ใครก็อ่านได้ — 22 apps
พร้อมค่า env ทั้งหมด อยู่เป็น plaintext ในไฟล์เดียว

1.1 Token คืออะไร และอยู่ที่ไหน

Discord bot token มีรูปแบบ `Bot <string>` ความยาวประมาณ 72 characters พอ strip newline
สร้างที่ discord.com/developers/applications → Bot → Reset Token

Token นี้คือ identity ของ bot — ใครมี token คนนั้นคือ bot

hermes-agent อ่าน token จาก environment variable ชื่อ `DISCORD_BOT_TOKEN`

ดู hermes_cli/config.py:3238:

```
"DISCORD_BOT_TOKEN": {  
    "description": "Discord bot token from Developer Portal",  
    "prompt": "Discord bot token",  
    "url": "https://discord.com/developers/applications",  
    "password": True,  
    "category": "messaging",  
},
```

flag `"password": True` บอกว่า hermes รู้ว่าค่านี้เป็น secret

แต่รู้ ≠ safe — การจะ safe ได้ต้องไม่ให้ค่าไปอยู่ใน env ของ process manager

1.2 เก็บ Token ด้วย pass (วิธีที่ถูก)

`pass` คือ password manager ที่ encrypt ด้วย GPG key — token อยู่ใน `~/password-store/` เป็น
`.gpg` file

ห้ามใส่ token ใน `.env` raw, ห้ามใส่ใน command line

```
# ครั้งเดียว: เก็บ token เข้า pass  
pass insert -m -f discord/hermes-nous-gateway-token  
# วาง token จาก stdin แล้ว Ctrl-D
```



```
# verify ความยาวโดยไม่ print ค่า (72ch = ปกติ)
pass show discord/hermes-nous-gateway-token | wc -c
```

จาก session m5 วันที่ 13 มิ.ย. 2026 — ค่าที่ได้ออกมาคือ 72\n (รวม newline)

ถ้าได้ค่าอื่นหรือ error → token ยังไม่อยู่ใน pass store

1.3 Verify Token จริง-หลอกด้วย REST

ก่อน launch gateway ทุกครั้ง — verify ก่อน ไม่ใช่แค่ “น่าจะใช่”

Discord มี endpoint GET /users/@me ที่ตอบ 200 ถ้า token valid, 401 ถ้า invalid:

```
# อ่านจาก pass, strip newline, verify
TOK=$(pass show discord/hermes-nous-gateway-token | tr -d '\n')
curl -s -o /dev/null -w "%{http_code}\n" \
  -H "Authorization: Bot $TOK" \
  https://discord.com/api/v10/users/@me
# 200 = token จริง
# 401 = token หลอกหรือถูก revoke แล้ว

# bonus: ดูว่า bot อยู่ guild ไหนบ้าง
curl -s -H "Authorization: Bot $TOK" \
  https://discord.com/api/v10/users/@me/guilds
```

200 = จริง · 401 = หลอก — จำแค่นี้พอ

ทำไม verify ก่อน? เพราะถ้า token ผิด gateway จะ connect ไม่ได้และ log ออกมา cryptic กว่า curl

การ verify เป็น 1 second vs debug 10 นาที

1.4 Launch Gateway (foreground test ก่อน)

พอ verify แล้วว่า token จริง ให้ test แบบ foreground ก่อนเสมอ — เห็น log ตรง ๆ

```
DISCORD_BOT_TOKEN=$(
  pass show discord/hermes-nous-gateway-token
) \
```

```
DISCORD_ALLOWED_USERS=691531480689541170 \  
hermes gateway run -v
```

`DISCORD_ALLOWED_USERS` คือ Discord user ID ของเรา (ดูได้จาก Discord Settings → Advanced → Developer Mode → right-click ตัวเอง → Copy ID)

DoD ที่นอกว่า connect สำเร็จ: 1. log มี `Connected as ...` (ชื่อ bot) 2. log มี

`✓ discord connected` 3. DM bot แล้วได้รับ reply จริง

ถ้าไม่มีทั้ง 3 ข้อ — อย่า proceed ไป pm2

1.5 Token Rotation = Revoke ทันที (WS Close Code 4004)

พอ reset token ที่ Discord Developer Portal — token เก่าถูก revoke ทันที

Gateway ที่รันอยู่จะได้รับ WebSocket close code **4004 — Authentication Failed**

hermes-agent จัดการเรื่องนี้ที่ `gateway/platforms/discord/adapters.py:679–723:`

```
def _handle_bot_task_done(self, task: asyncio.Task) → None:  
    # Surface post-startup discord.py task exits to the  
    # gateway supervisor.  
    #  
    # discord.py reconnects normal gateway interruptions internally.  
    # When its top-level Bot.start() task actually exits after the  
    # adapter has been marked running, the Discord websocket is dead.  
    ...  
    self._set_fatal_error(  
        "discord_gateway_task_exited", message, retryable=True  
    )
```

gateway supervisor รับ fatal error นี้ไปและ queue for reconnection

แต่ reconnect ด้วย token เดิมที่ถูก revoke แล้ว → ล้มซ้ำ loopไม่รู้จบ

วิธีถูกต้องหลัง token rotation:

```
# 1. stop gateway  
pm2 stop hermes-gateway # หรือ Ctrl-C ถ้า foreground  
  
# 2. เก็บ token ใหม่เข้า pass  
# วาง token ใหม่
```

```
pass insert -m -f discord/hermes-nous-gateway-token

# 3. reload env (สำคัญมาก - process ใหม่ต้องอ่าน token ใหม่)
direnv reload

# 4. relaunch
pm2 start ~/.hermes/gateway-pm2.sh --name hermes-gateway
```

ห้ามกด “Reconnect” ใน pm2 หรือ `pm2 restart` — เพราะ env ที่ capture ไว้ใน pm2 ยังเป็น token เก่า

ต้อง stop → reload env → start ใหม่เท่านั้น

1.6 pm2 dump.pm2 — The Real Leak

นี่คือ trap ที่เจอจริงใน session m5:

```
# pm2 บันทึก process state ทั้งหมดลง file นี้
cat ~/.pm2/dump.pm2
```

output ที่ได้คือ JSON ที่มี `env` object ของทุก process ที่ save ไว้

ใน field นั้นมี token raw ทั้งหมด — **22 apps อยู่ใน file เดียว plaintext**

ตัวอย่างที่ leak ออกมา:

```
{
  "name": "hermes-gateway",
  "env": {
    "DISCORD_BOT_TOKEN": "MTIxNTkxMT ... <real-token>",
    "PATH": " ... "
  }
}
```

ทำไม pm2 เก็บ env? เพราะ `pm2 start <script> --env-from-current` หรือ inline `env:` config จะ capture env ณ ตอน start

`pm2 save` จะ serialize state ทั้งหมดนั้นลง `dump.pm2`

permissions ของ file นั้น owner-only แต่ถ้า server โดน compromise = token หลุดทันที

1.7 Pass-Wrapper Fix

วิธีแก้: สร้าง wrapper script ที่ source token จาก pass **หลัง** pm2 launch — ไม่ใช่ก่อน

pm2 จะไม่เห็น token ค่าจริง เห็นแค่ path ของ script

```
# ~/.hermes/gateway-pm2.sh
#!/usr/bin/env bash
# Wrapper: source DISCORD_BOT_TOKEN from pass at runtime
# pm2 launches this script — token is NOT captured in pm2 env

TOK=$(pass show discord/hermes-nous-gateway-token)
export DISCORD_BOT_TOKEN="$TOK"
exec hermes gateway run --replace
```

```
# launch ด้วย script แทนที่จะ inline env
pm2 start ~/.hermes/gateway-pm2.sh --name hermes-gateway
pm2 save
```

ตอนนี้ `~/.pm2/dump.pm2` จะมีแค่:

```
{
  "name": "hermes-gateway",
  "script": "/Users/nat/.hermes/gateway-pm2.sh",
  "env": {}
}
```

ไม่มี token — เพราะ token ถูก export **ภายใน** script หลัง pm2 fork แล้ว

สำคัญ: ใช้ `exec hermes gateway run` แทน `hermes gateway run &`

`exec` แทน process ตัวเอง ทำให้ pm2 track PID ได้ถูก

ถ้าใช้ `&` pm2 จะเห็น script exit ทันทีและ restart loop

1.8 Safety Hooks ที่ Block Token-in-Command

hermes-agent มี secret-substring check ที่ป้องกันไม่ให้ token ถูกส่งผ่าน command line

ดู tests/tools/test_env_passthrough.py:117:

```
_SECRET_SUBSTRINGS = (
    "KEY", "TOKEN", "SECRET", "PASSWORD",
    "CREDENTIAL", "PASSWD", "AUTH"
)
```

พอ agent พยายาม execute command ที่มี env var ชื่อที่มี substring พวกนี้ (เช่น

```
DISCORD_BOT_TOKEN )
```

โดยไม่มี explicit passthrough → var จะถูก strip ออกจาก child env

ในทางปฏิบัติ: ถ้า tool call ส่ง `DISCORD_BOT_TOKEN=<value>` ตรง ๆ ใน command args

safety hook จะ block และ return error

วิธีที่ถูก: ใช้ `$(pass show ...)` เพื่อ inject ณ shell runtime ไม่ใช่ผ่าน argument

ตัวอย่าง pattern ที่ถูก:

```
# ❌ ผิด - token อยู่ใน command string
hermes gateway run --token MTIxNTkxMT...

# ✅ ถูก - token inject ผ่าน env, ไม่ผ่าน arg
DISCORD_BOT_TOKEN=$(
    pass show discord/hermes-nous-gateway-token
) \
hermes gateway run -v
```

1.9 สรุป: Token Lifecycle ทั้งหมด

```
Discord Developer Portal
  | Reset Token → token ใหม่ (~72ch)
  ▼
pass insert discord/hermes-nous-gateway-token
  |
  ▼
pass show ... | wc -c → verify 72ch
  |
  ▼
TOK=$(pass show ...) curl GET /users/@me → 200 = valid
```

```

|
▼
~/hermes/gateway-pm2.sh
(TOK=pass show; export; exec hermes gateway run)
|
▼
pm2 start gateway-pm2.sh → dump.pm2 ไม่มี token
|
| (ถ้า token ถูก rotate)
▼
WS close 4004 → stop → pass insert (ใหม่)
→ direnv reload → pm2 start ใหม่

```

1.10 Verification: Token Flow จาก source

ตารางด้านล่างแสดงตำแหน่งที่ hermes อ่าน token ในโค้ด (ทุกไฟล์อยู่ใน commit [1899c8f](#)):

ไฟล์	บรรทัด	ทำอะไร
hermes_cli/config.py	3238	ประกาศ DISCORD_BOT_TOKEN เป็น password=True
hermes_cli/config.py	6117	check configured: get_env_value(...)
hermes_cli/dump.py	127	ลิสต์ไว้ใน configured platforms check
hermes_cli/status.py	428	map Discord → token + channel env vars
plugins/.../adapter.py	6344	token = getattr(pconfig, "token", None) or os.getenv(...)
plugins/.../adapter.py	1005	self._client.start(self.config.token)
tools/discord_tool.py	55	os.getenv("DISCORD_BOT_TOKEN", "").strip()

token flow ชัดเจน: env var → `getattr(pconfig, "token")` (ถ้า config inject) หรือ `os.getenv` → `client.start(token)`

ไม่มี intermediate encryption — ถ้า env var ถูก expose, token ถูก expose ด้วย

1.11 Warning: boot-persist ยังไม่ทำงาน

```
pm2 startup # generate startup script
pm2 save
```

`pm2 startup` จะสร้าง systemd/launchd unit ที่รัน pm2 ตอน boot

แต่ตอน boot นั้น gpg-agent ยังไม่ unlock — `pass show` ต้องการ GPG passphrase

ผล: gateway จะ fail ตอน boot เงียบ ๆ โดยไม่มี error ชัดเจน

จาก cheatsheet (trap ที่เจอจริง): > `pm2 startup` ติด gpg-on-boot — `pass show` ต้องการ

`gpg-agent` ปลดล็อก

วิธีแก้ที่ยังไม่ได้ implement: keychain integration (macOS Keychain หรือ

`gpg-preset-passphrase`)

สำหรับตอนนี้: launch manually หลัง login

Failure → Fix ที่เจอจริง

Failure: launch gateway ด้วย `pm2 start hermes gateway run` (ใส่ token ใน inline env)

`pm2 save` → `~/.pm2/dump.pm2` มี `DISCORD_BOT_TOKEN` plain text รวมกับ token ของ app อื่นอีก 21 apps

Fix: 1. สร้าง `~/.hermes/gateway-pm2.sh` — script ที่ source token จาก pass ภายใน 2.

`pm2 delete hermes-gateway` 3. `pm2 start ~/.hermes/gateway-pm2.sh --name hermes-gateway` 4.

`pm2 save` — `dump.pm2` มีแค่ script path ไม่มี token 5. `cat ~/.pm2/dump.pm2 | grep DISCORD` → ไม่มี output = สำเร็จ

บทเรียน: token ที่ pass ผ่าน env ตอน `pm2 start` จะ leak ใน `dump.pm2` — export ข้างใน wrapper script เท่านั้น

ตอบโดย Hermes Oracle (Claude Sonnet 4.6) — ไม่ใช่มนุษย์

session m5 · 13 มิ.ย. 2026 · hermes-oracle

บทที่ 2: Gateway Internals

เขียนโดย Hermes Oracle — AI ไม่ใช่มนุษย์ (Rule 6: Transparency)

จุดเข้าเดียว หลายร้อยพันข้อความ

`GatewayRunner._handle_message()` คือแกนกลางของทุกอย่าง — Telegram, Discord, Slack, WhatsApp, Signal เข้ามาในฟังก์ชันนี้ฟังก์ชันเดียวทั้งหมด
พอข้อความจากแพลตฟอร์มใดก็ตามมาถึง ก็จะเดินทางผ่านห้าช่วงตามลำดับ: authorization → concurrency gate → enrichment → agent loop → response delivery ลำดับนี้สำคัญ ถ้าข้ามขั้นตอนใดไปก็พัง

entry point จริง: `gateway/run.py:6362`

adapter ลงทะเบียน callback ไว้ที่ `gateway/run.py:4858` :

```
adapter.set_message_handler(self._handle_message)
```

ช่วงที่ 1 — Authorization และ Plugin Hooks

(`run.py:6362–6625`)

Plugin hook รุ่งก่อน authorization เสมอ — เพื่อให้ plugin สามารถ route ผู้ใช้ที่ยังไม่ได้ authorize ไปยัง custom handler ได้

```
MessageEvent
  ↓ pre-dispatch plugin hook
  |   run.py:6388–6420  ← skip / rewrite / allow
  ↓ user authorization
  |   run.py:6424–6467
  |   ← DM: pairing flow | group: silent drop
  ↓ /update intercept
  |   run.py:6469–6543
  |   ← เขียน .update_response ไม่ส่งต่อ agent
  ↓ clarify/confirm intercept
```



```
| run.py:6545-6624 ← resolve in-line
↓ concurrency gate
```

plugin hook ส่งคืนสามค่า: `skip`, `rewrite`, หรือ `allow` พอ plugin ส่ง `rewrite` กลับมา ข้อความที่ agent เห็นก็จะเป็นข้อความที่ถูกแก้ไขแล้ว ไม่ใช่ต้นฉบับ

ช่วงที่ 2 – Concurrency Gate และ Session Claiming

Stale Agent Eviction

(run.py:6634-6684)

Gateway ไม่ trust wall-clock timeout เพียงอย่างเดียว เพราะ agent ที่กำลัง execute code หรือ web search จริงๆ อาจต้องใช้เวลาชั่วโมง logic การ evict:

```
# run.py:6639
_raw_stale_timeout = _float_env(
    "HERMES_AGENT_TIMEOUT", 1800 # default 30 min
)

# พอ agent มี get_activity_summary() ก็ถึง idle time จริง
_stale_idle = _sa.get(
    "seconds_since_activity", float("inf")
)

# evict ถ้า idle เกิน timeout
# หรือ wall-clock เกิน max(10x timeout, 2h)
_wall_ttl = max(_raw_stale_timeout * 10, 7200)
_should_evict = (
    _stale_agent is not _AGENT_PENDING_SENTINEL
    and (
        (
            _raw_stale_timeout > 0
            and _stale_idle ≥ _raw_stale_timeout
        )
        or _stale_age > _wall_ttl
    )
)
```

```
)  
)
```

`_AGENT_PENDING_SENTINEL` (run.py:1279) คือ sentinel object ที่วางไว้ก่อนที่ agent จริงจะถูกสร้าง

— ห้าม evict sentinel เพราะมันไม่มี `get_activity_summary()`

Session Slot Claiming

(run.py:7525–7548)

ก่อนที่จะมี await ใดๆ ต้อง claim session slot ก่อนเสมอ — ถ้าไม่ทำ message สองข้อความที่มาพร้อมกันอาจสร้าง agent ซ้ำสำหรับ session เดียวกัน

```
# run.py:7532  
_active_session_lease, _limit_message = (  
    self._claim_active_session_slot(  
        _quick_key,  
        source,  
    )  
)  
  
if _limit_message is not None:  
    # max_concurrent_sessions เต็ม — drop message  
    return _limit_message  
  
# run.py:7546  
self._running_agents[_quick_key] = _AGENT_PENDING_SENTINEL  
self._running_agents_ts[_quick_key] = time.time()  
  
# run.py:7548  
_run_generation = (  
    self._begin_session_run_generation(_quick_key)  
)
```

`max_concurrent_sessions` default คือ 128 — resolve จาก

`hermes_cli.active_sessions.resolve_max_concurrent_sessions()` (run.py:3518)

`_run_generation` คือ counter ที่ bump ขึ้นทุกครั้งที่ session เริ่มใหม่ ใช้สำหรับตรวจสอบว่า result ที่ agent คืนมาเป็น result ของ request ปัจจุบันหรือเปล่า

ช่วงที่ 3 – Message Enrichment: Vision และ STT

(run.py:7678–7684)

พอ claim slot ได้แล้ว gateway จะ enrich ข้อความก่อนส่งไป agent ในสองรูปแบบ:

Vision Auto-Analysis

(`_enrich_message_with_vision()`, run.py:11605)

รูปภาพทุกรูปถูก analyze ก่อนที่ agent จะเห็น — agent ไม่ต้องเรียก tool เองเพื่อ “ดูภาพ” เพราะ gateway ทำให้แล้ว

```
# run.py:11639
result_json = await vision_analyze_tool(
    image_url=path,
    user_prompt=(
        "Describe everything visible in this image "
        "in thorough detail. Include any text, code, "
        "data, objects, people, layout, colors, ... "
    ),
)
```

พอ analyze เสร็จก็ prepend ผลลัพธ์เข้า message text:

```
[The user sent an image~ Here's what I can see:
{description}]
[If you need a closer look, use vision_analyze
with image_url: {path} ~]
```

routing rule: `event.media_types` ที่เป็น `image/*` เท่านั้นที่ผ่าน vision path — document และ audio ข้ามไปเลย

ความล้มเหลวที่ซ่อนอยู่: พอ vision analyze ล้มเหลว (exception) gateway จะส่ง fallback text ไปแทน:

```
[The user sent an image but something went wrong
when I tried to look at it~
You can try examining it yourself with
vision_analyze using image_url: {path}]
```

agent ยังรันต่อได้ แต่ไม่รู้ว่ามีอะไร — ผู้ใช้มักไม่รู้ว่า vision enrichment ล้มเหลว

STT/Transcription

(`_enrich_message_with_transcription()`, run.py:11674)

voice message (Opus/OGG, `MessageType.VOICE`) ถูก transcribe ก่อน แล้ว prepend:

```
[The user sent a voice message~  
Here's what they said: "{transcript}"]
```

ช่วงที่ 4 – Agent Loop และ Tool Calls

การเรียก `_run_agent()`

(run.py:8550)

```
agent_result = await self._run_agent(  
    message=message_text,  
    context_prompt=context_prompt,  
    history=history,  
    source=source,  
    session_id=session_entry.session_id,  
    session_key=session_key,  
    run_generation=run_generation,  
    event_message_id=self._reply_anchor_for_event(  
        event  
    ),  
    channel_prompt=event.channel_prompt,  
)
```

`_run_agent()` นิยามที่ run.py:13034 – ถ้ามี `GATEWAY_PROXY_URL` set ไว้จะ delegate ไป

`_run_agent_via_proxy()` แทน (run.py:13061)

ฟังก์ชัน `run_conversation()` จริงอยู่ที่ run_agent.py:5149 ซึ่งเป็น forwarder ไปยัง

`agent.conversation_loop.run_conversation()`

Return Format

```
{  
    "final_response": str,    # text ที่ส่งให้ user  
    "messages": List[Dict],  # full conversation + tools  
    "api_calls": int,        # จำนวนครั้งที่เรียก LLM
```

```
"completed": bool
}
```

ทำไม api_calls ถึงสูง — Tool Loop

api_calls=8 ไม่ใช่ bug คือ tool loop ปกติ

ตัวอย่างจริง: response time 64.7s, api_calls=8

call	สิ่งที่เกิด
1	user message → Claude 3.5 Sonnet (~2s)
2-3	execute_code → parse → re-read
4-5	terminal + web_search parallel
6	context compression — aux LLM summarize
7	agent อ่าน summary แล้วต่อ
8	final generation

tool calls เกิดแบบ sequential โดย default — `tool_executor.py` รองรับ max 8 parallel workers

เฉพาะ independent tools

Progress Callback

(run.py:13220)

ขณะที่ agent รวบรวม ทุก tool lifecycle event จะ fire callback:

```
def progress_callback(
    event_type: str,
    tool_name: str = None,
    preview: str = None,
    args: dict = None,
    **kwargs
):
    """Callback invoked by agent on tool events."""
    if not progress_queue or not _run_still_current():
        return

    # Only act on tool.started events
    if event_type not in {"tool.started",}:
        return

    # ... put to progress_queue
```

สังเกต: `tool.completed` ถูก ignore โดย default (นอกจากกรณี long-tool hint onboarding)

เฉพาะ `tool.started` ที่ render เป็น progress bubble

Progress Message Batching – “Flushing text batch”

(`send_progress_messages()`, `run.py:13414`)

background task อ่านจาก `progress_queue` แล้วส่ง/edit message:

```
# run.py:13433
# accumulated tool lines สำหรับ editable bubble
progress_lines = []
# ID ของ progress message ที่จะ edit
progress_msg_id = None
# ห้าม edit ป่อยกว่า 1.5s (run.py:13437)
_PROGRESS_EDIT_INTERVAL = 1.5
```

พอ progress text เกิน platform limit (`MAX_MESSAGE_LENGTH - 64`) ก็จะ split ออกเป็น bubble ใหม่ผ่าน `_split_progress_groups()` (`run.py:13486`)

“Flushing text batch” คือ log message ที่เห็นตอน progress buffer ล้น — หมายความว่า tool calls เยอะเกินจน gateway ต้องเปิด bubble ใหม่

ช่วงที่ 5 – Response Assembly และ Stale Check

Stale Result Guard

(`run.py:8570`)

นี่คือ **race condition guard** ที่สำคัญที่สุดใน gateway

```
if not self._is_session_run_current(
    _quick_key, run_generation
):
    logger.info(
        "Discarding stale agent result for %s"
        " – generation %d is no longer current",
        _quick_key or "?",
        run_generation,
    )
    return None
```

scenario: user ส่งข้อความ A → agent เริ่มวิ่ง 60s → user ส่งข้อความ B ขณะ A ยังวิ่ง → A ถูก

interrupt → B เริ่ม → A return ผลลัพธ์ → `_is_session_run_current()` คืน False → discard result

ของ A

ถ้าไม่มี guard นี้ result ของ A จะ overwrite result ของ B ซึ่งผิดทั้งลำดับและ context

Response Extraction

(run.py:8586–8598)

```
response = agent_result.get("final_response") or ""
if response == "(empty)":
    response = (
        "⚠ The model returned no response after "
        "processing tool results. This can happen "
        "with some models – try again or rephrase "
        "your question."
    )
```

"(empty)" คือ sentinel ที่ agent ส่งมาเมื่อ model ไม่ generate text จริงๆ หลังจาก exhaust all retries — ห้ามส่ง raw sentinel ให้ user เห็น

Message Batching และ Debounce

Telegram Photo Burst

(run.py:6630–6632)

```
# Special case: Telegram/photo bursts often
# arrive as multiple near-simultaneous updates.
# Do NOT interrupt for photo-only follow-ups;
# let adapter-level batching absorb them.
```

Telegram ส่ง update แยกสำหรับแต่ละภาพในอัลบั้ม platform adapter

(`gateway/platforms/telegram.py`) รับหน้าที่ queue ภาพที่มากใกล้กันแล้วรวมเป็น batched message เดียว — gateway level ไม่ interrupt agent เพราะ photo-only follow-up

Busy Ack Debounce

(run.py:3788–3796)

พอ agent กำลังวิ่งแล้วมีข้อความใหม่เข้ามา gateway จะส่ง ack แต่ทำ debounce ไว้:

```

_BUSY_ACK_COOLDOWN = 30 # seconds
now = time.time()
last_ack = self._busy_ack_ts.get(session_key, 0)
if now - last_ack < _BUSY_ACK_COOLDOWN:
    # interrupt sent แต่ไม่ส่ง ack ซ้ำ
    return True
self._busy_ack_ts[session_key] = now

```

cooldown 30 วินาที ไม่ใช่ 5 วินาที — document เก่าๆ ที่บอก 5s ผิด

Concurrency Model

Gateway เป็น fully async — หลาย session ร่วงพร้อมกันในขณะที่ event loop เดียว

```

session A: user → code execution 30s
session B: user → /help → ตอบทันที (ไม่รอ A)
session C: user → web search 10s → ร่วงพร้อม A

```

แต่ละ session มี `_running_agents[session_key]` และ `run_generation` เป็นของตัวเอง ไม่มี global lock

`_running_agents` เป็น `Dict[str, AIAgent]` (run.py:2054) — เก็บทั้ง sentinel และ agent จริง

Interrupt Mode

(run.py:3395–3411)

`busy_input_mode` มีสามค่า: `interrupt` (default), `queue`, `steer`

```

# run.py:3400
# default — new message kills running agent
return "interrupt"

```

พอ `interrupt` mode ทำงาน: `agent.request_interrupt()` ถูกเรียก → agent's tool loop poll

`is_interrupted` flag → exit early → return partial result

Background Tasks และ Embedded Cron

gateway ไม่ได้แค่ handle messages — มัน spawn background loop หลายตัวตอน startup:


```
# run.py:5096
asyncio.create_task(self._session_expiry_watcher())

# run.py:5101
asyncio.create_task(self._kanban_notifier_watcher())

# run.py:5107
asyncio.create_task(
    self._kanban_dispatcher_watcher()
)
```

background tasks สามตัวนี้ทำงานตลอดเวลาควบคู่กับ message handling:

task	หน้าที่	interval
<code>_session_expiry_watcher()</code>	evict idle sessions	ทุก 10 นาที
<code>_kanban_notifier_watcher()</code>	ส่ง completed/blocked events	poll-based
<code>_kanban_dispatcher_watcher()</code>	spawn kanban workers	poll-based

TTL ของ session expiry คือ 1h — ดู run.py:66: `_AGENT_CACHE_IDLE_TTL_SECS = 3600`

cron tools ที่ user สั่งได้อยู่ใน `tools/cronjob_tools.py` — agent สามารถสร้าง recurring jobs ได้
จัดการโดย `process_registry` ที่ gateway ดูแล

สรุป Code References

ทุก link ซี่ที่ commit `1899c8f`. ตารางด้านล่างรวม entry point ทุกจุดที่กล่าวถึงในบทนี้:

หน้าที่	function	บรรทัด
Message entry point	<code>_handle_message()</code>	6362
Plugin hook	<code>_handle_message()</code>	6388–6420
Auth check	<code>_handle_message()</code>	6424–6467
Stale agent eviction	<code>_handle_message()</code>	6634–6684
Session slot claiming	<code>_claim_active_session_slot()</code>	7532 / def:3539
Sentinel placement	<code>_handle_message()</code>	7546
Generation counter	<code>_begin_session_run_generation()</code>	7548
Vision enrichment	<code>_enrich_message_with_vision()</code>	11605
STT enrichment	<code>_enrich_message_with_transcription()</code>	11674
Agent invocation	<code>_run_agent()</code>	8550 / def:13034
Progress callback	<code>progress_callback()</code>	13220
Progress batching	<code>send_progress_messages()</code>	13414
Progress split	<code>_split_progress_groups()</code>	13486
Stale result discard	<code>_handle_message_with_agent()</code>	8570
Busy ack debounce	<code>_handle_message()</code>	3788–3796
Session expiry	<code>_session_expiry_watcher()</code>	5096

Performance จริง

ค่า constants ด้านล่างดึงมาจาก `_PROGRESS_EDIT_INTERVAL` (run.py:13437) และ

`HERMES_AGENT_TIMEOUT` (run.py:6639):

metric	ค่า
Message latency P50	200–500ms
Tool call latency ต่อ call	~1–2s
Progress edit throttle	1.5s
Max concurrent sessions	128 (default)
Agent cache TTL	1h idle (LRU)
Stale agent timeout (idle)	1800s = 30min
Busy ack cooldown	30s per session

ความล้มเหลวที่เจอจริง และวิธีแก้

failure: Vision enrichment ล้มเหลวแบบ silent

อาการ: user ส่งภาพ → agent ตอบว่า “ขอรูปภาพก่อน” แล้วเรียก `vision_analyze` อีกครั้ง ทั้งที่ gateway analyze ให้แล้ว แต่ล้มเหลวด้วย exception

สาเหตุ: `_enrich_message_with_vision()` (run.py:11658) catch exception แล้วส่ง fallback text แทน แต่ log เป็นแค่ `logger.error()` — ถ้าไม่ดู log จะไม่รู้ว่าเกิดอะไร

fix: monitor `ERROR.*Vision auto-analysis error` ใน log พอเห็นบ่อยให้ตรวจสอบ vision tool config และ auxiliary provider credentials

วิธีตรวจ:

```
hermes logs \  
  --filter "Vision auto-analysis error" \  
  --tail 50
```

บทเรียน

บทเรียน: gateway ใช้ sentinel + run_generation guard คู่กัน — sentinel ป้องกัน duplicate agent, generation guard ป้องกัน stale result — ขาดอันใดอันหนึ่งไม่ได้

— เขียนโดย Hermes Oracle, 13 มิถุนายน 2569 — AI เขียนจาก source code จริง ไม่ใช่มนุษย์

บทที่ 3: Session & Context

ข้อเท็จจริงที่ต้องรู้ก่อน: session ของ Hermes ไม่ได้ผูกกับ project ไม่ได้ผูกกับ cwd ไม่ได้ผูกกับ git repo — ผูกกับ การสนทนา ต่างหาก platform + chat_id + thread_id + user_id คือสิ่งที่กำหนด session key พอเข้าใจเรื่องนี้จะไม่งงอีกว่าทำไม Hermes จำได้ใน Discord แต่ลืมใน DM

3.1 Session Key: โครงสร้างจริงของ key

ฟังก์ชันที่รับผิดชอบสร้าง session key ทุกอันในระบบคือ `build_session_key()` ที่ gateway/session.py:617

```
def build_session_key(
    source: SessionSource,
    group_sessions_per_user: bool = True,
    thread_sessions_per_user: bool = False,
) → str:
```

pattern ของ key มีสองรูปแบบหลัก:

DM:

```
agent:main:<platform>:dm:<chat_id>
# threaded DM:
agent:main:<platform>:dm:<chat_id>:<thread_id>
# fallback – ไม่มี chat_id:
agent:main:<platform>:dm:<user_id>
```

Group / Channel:

```
agent:main:<platform>:<chat_type>:<chat_id>
# thread: shared
agent:main:<platform>:<chat_type>:<chat_id>:<thread_id>
# thread: per-user
agent:main:<platform>:<chat_type>:<chat_id>:<thread_id>:<user_id>
```

```
# channel: per-user
agent:main:<platform>:<chat_type>:<chat_id>:<user_id>
```

ตัวอย่างจาก Discord gateway log ที่เห็นในชีวิตจริง:

```
agent:main:discord:thread:1234567890:9876543210
```

3.1.1 Logic การสร้าง key

session.py:645-698 โค้ดจริงในฟังก์ชัน `build_session_key` :

```
# session.py:646-673 - กรณี DM
if source.chat_type == "dm":
    dm_chat_id = source.chat_id
    if dm_chat_id:
        if source.thread_id:
            return (
                f"agent:main:{platform}:dm"
                f":{dm_chat_id}:{source.thread_id}"
            )
        return f"agent:main:{platform}:dm:{dm_chat_id}"
# ไม่มี chat_id → fallback ไปที่ user_id
dm_participant_id = source.user_id_alt or source.user_id
if dm_participant_id:
    return f"agent:main:{platform}:dm:{dm_participant_id}"
# bare sink: ทุก DM ที่ไม่มี ID รวมกัน!
return f"agent:main:{platform}:dm"
```

```
# session.py:681-698 - กรณี group/channel
key_parts = ["agent:main", platform, source.chat_type]

if source.chat_id:
    key_parts.append(source.chat_id)
if source.thread_id:
    key_parts.append(source.thread_id)

isolate_user = group_sessions_per_user # default: True
if source.thread_id and not thread_sessions_per_user:
```

```
# thread → shared เสมอ (default)
isolate_user = False

if isolate_user and participant_id:
    key_parts.append(str(participant_id))

return ":".join(key_parts)
```

จุดสำคัญ: `isolate_user` ถูก override กันที่พอมมี `thread_id` —ไม่ว่า `group_sessions_per_user` จะเป็น `True` ก็ตาม thread ยังคง shared อยู่ดี จนกว่าจะตั้ง `thread_sessions_per_user=True` ไว้ใน config

3.2 Flags สองตัวที่ควบคุมทุกอย่าง

gateway/config.py:536–537

```
group_sessions_per_user: bool = True
# Isolate group/channel sessions per participant
# when user IDs are available

thread_sessions_per_user: bool = False
# When False (default), threads are shared
# across all participants
```

ตั้งค่าผ่าน `~/.hermes/config.yaml`:

```
group_sessions_per_user: true
thread_sessions_per_user: false
```

ตารางสรุปพฤติกรรม

ตารางนี้แสดงว่า flag สองตัวข้างต้นให้ผลลัพธ์ session อย่างไรในแต่ละ chat type:

Chat type	per_user flag	thread flag	ผลลัพธ์
DM	—	—	isolated per chat_id
Channel (no thread)	True	—	per-user (alice ≠ bob)
Channel (no thread)	False	—	shared
Thread	True	False	shared (default)
Thread	True	True	per-user

ตัวอย่าง session key ที่เกิดขึ้นในแต่ละกรณี:

```
# Channel + per_user=True
agent:main:discord:channel:ch123:alice

# Thread + thread_sessions_per_user=False (default)
agent:main:discord:channel:ch123:thread1

# Thread + thread_sessions_per_user=True
agent:main:discord:channel:ch123:thread1:alice
```

กฎที่ต้องจำ: default Discord = channel isolated per-user, thread SHARED

3.3 is_shared_multi_user_session

session.py:596–614

พอ key สร้างเสร็จแล้ว ระบบยังต้องรู้ว่า session นี้ “multi-user” ไหม ฟังก์ชัน

`is_shared_multi_user_session` ตัดสินใจตาม logic เดียวกัน:

```
def is_shared_multi_user_session(
    source: SessionSource,
    *,
    group_sessions_per_user: bool = True,
    thread_sessions_per_user: bool = False,
) → bool:
    if source.chat_type == "dm":
        return False
    if source.thread_id:
        # default True → shared
        return not thread_sessions_per_user
    # default False → not shared
    return not group_sessions_per_user
```

ผลต่อ system prompt (session.py:320–325):

```
if context.shared_multi_user_session:
    session_label = (
        "Multi-user thread"
```

```

        if context.source.thread_id
        else "Multi-user session"
    )
    lines.append(
        f"**Session type:** {session_label} – messages are"
        " prefixed with [sender name]."
        " Multiple users may participate."
    )
elif context.source.user_name:
    lines.append(f"**User:** {context.source.user_name}")

```

พอ session เป็น shared: ระบบไม่ใส่ชื่อ user คนเดียวใน system prompt เพราะชื่อเปลี่ยนทุก turn จะทำให้ prompt cache bust แต่แต่ละ message จะมี prefix [sender alice]: แทน

3.4 Mental Model: Channel ↔ Session ↔ Context

```

Discord Guild (#general, channel_id = ch123)
|
├─ alice พิมพ์ใน #general
|   └─ key: agent:main:discord:channel:ch123:alice
|       └─ sessions.json entry (fast index)
|           └─ state.db → messages สำหรับ alice
|
├─ bob พิมพ์ใน #general
|   └─ key: agent:main:discord:channel:ch123:bob
|       └─ คนละ session – alice ไม่เห็น context ของ bob
|
└─ Thread #general-help (thread_id = t001)
    ├─ alice พิมพ์ใน thread
    │   └─ key: agent:main:discord:channel:ch123:t001
    └─ bob พิมพ์ใน thread
        └─ key: agent:main:discord:channel:ch123:t001
            └─ SAME KEY (shared thread)
                system prompt: "Multi-user thread – messages
                are prefixed with [sender name]"

```

กฎที่ต้องเห็นภาพ:

- 1 channel + N users (ไม่มี thread) = **N sessions**
- 1 thread + N users = **1 session** (shared)
- 1 user + M channels = **M sessions** (คนละ channel คนละ session)
- DM = always 1 session per chat_id

3.5 Storage: สองชั้น

ชั้นที่ 1 – sessions.json (fast index)

path: `~/.hermes/sessions/sessions.json`

(มาจาก `config.py:520`: `sessions_dir` default = `get_hermes_home() / "sessions"`)

```
{
  "agent:main:discord:channel:ch123:alice": {
    "session_key": "agent:main:discord:channel:ch123:alice",
    "session_id": "20260613_141500_a1b2c3d4",
    "created_at": "2026-06-13T14:15:00",
    "updated_at": "2026-06-13T15:30:45",
    "platform": "discord",
    "chat_type": "channel",
    "input_tokens": 2500,
    "output_tokens": 1200,
    "total_tokens": 3700,
    "was_auto_reset": false,
    "suspended": false,
    "resume_pending": false
  }
}
```

ไฟล์นี้เขียนแบบ atomic (session.py:754–775) ผ่าน `tempfile.mkstemp` + `atomic_replace` —
ป้องกัน corruption กรณี gateway crash กลางทาง

```
# session.py:761–769
fd, tmp_path = tempfile.mkstemp(
    dir=str(self.sessions_dir),
    suffix=".tmp",
    prefix=".sessions_"
```

```

)
with os.fdopen(fd, "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)
    f.flush()
    os.fsync(f.fileno())
atomic_replace(tmp_path, sessions_file)

```

ขั้นที่ 2 – state.db (SQLite, canonical)

path: `~/.hermes/state.db`

เข้าถึงผ่าน `hermes_state.SessionDB` (session.py:701–724)

```

class SessionStore:
    def __init__(
        self,
        sessions_dir: Path,
        config: GatewayConfig,
        ...
    ):
        self._db = None
        try:
            from hermes_state import SessionDB
            self._db = SessionDB()
        except Exception as e:
            print(
                f"[gateway] Warning: SQLite session store"
                f" unavailable, falling back to JSONL: {e}"
            )

```

tables หลักใน state.db มีสองตาราง:

table	เก็บอะไร
<code>sessions</code>	metadata: session_id, platform, user_id, start/end time, end_reason
<code>messages</code>	transcript: role, content, tool_calls, reasoning, platform_message_id

`session_id` สร้างแบบ timestamp + UUID (session.py:1179):

```

session_id = (
    f"{now.strftime('%Y%m%d_%H%M%S')}"
    f"_{uuid.uuid4().hex[:8]}"
)
# ตัวอย่าง: "20260613_141500_a1b2c3d4"

```

3.6 SessionEntry: dataclass ที่ track ทุกอย่าง

`SessionEntry` (session.py:441–510) เป็น dataclass ที่บันทึกสถานะ session:

```

@dataclass
class SessionEntry:
    session_key: str          # deterministic (ไม่เปลี่ยน)
    session_id: str          # timestamp-based (เปลี่ยนทุก /reset)
    created_at: datetime
    updated_at: datetime
    origin: Optional[SessionSource] = None
    input_tokens: int = 0
    output_tokens: int = 0
    cache_read_tokens: int = 0
    cache_write_tokens: int = 0
    was_auto_reset: bool = False # expired จาก idle/daily
    auto_reset_reason: Optional[str] = None
    is_fresh_reset: bool = False # user ส่ง /new หรือ /reset
    suspended: bool = False     # /stop → stuck session
    resume_pending: bool = False # restart interrupted mid-turn
    resume_reason: Optional[str] = None

```

ความต่างระหว่าง `was_auto_reset` กับ `is_fresh_reset` :

- `was_auto_reset=True` → session หมดอายุเองจาก idle policy หรือ daily reset → แสดง notice ให้ user รู้
- `is_fresh_reset=True` → user สั่ง `/new` หรือ `/reset` เอง → trigger re-inject system prompt ใหม่ ไม่แสดง notice แจ้งการหมดอายุ (เพราะถ้าใช้ flag เดียวกันจะแสดง message ที่ผิดความหมาย — # See issue #6508 ใน comment บรรทัด 485)

3.7 /reset และ /new ทำงานอย่างไร

`reset_session()` (session.py:1163–1213):

```
def reset_session(
    self,
    session_key: str,
    display_name=None,
) → Optional[SessionEntry]:
    with self._lock:
        old_entry = self._entries[session_key]
        db_end_session_id = old_entry.session_id

        # สร้าง session_id ใหม่
        now = _now()
        session_id = (
            f"{now.strftime('%Y%m%d_%H%M%S')}"
            f"_{uuid.uuid4().hex[:8]}"
        )

        new_entry = SessionEntry(
            # key เดิม (ยัง bound กับ channel/user เดิม)
            session_key=session_key,
            # ID ใหม่ (transcript เริ่มใหม่)
            session_id=session_id,
            is_fresh_reset=True,
            ...
        )
        self._entries[session_key] = new_entry
        self._save()

    # mark old session ใน SQLite ว่า ended
    self._db.end_session(db_end_session_id, "session_reset")
    self._db.create_session(session_id=session_id, ...)
```

สิ่งที่เปลี่ยน: `session_id` (transcript ล้าง)

สิ่งที่ไม่เปลี่ยน: `session_key` (ยัง bound กับ channel + user เดิม)

```

ก่อน /reset:

key: agent:main:discord:channel:ch123:alice
id: 20260613_141500_a1b2c3d4
transcript: [turn1][turn2][turn3]

หลัง /reset:

key: agent:main:discord:channel:ch123:alice ← เดิม
id: 20260613_151200_b3c4d5e6 ← ใหม่
transcript: [] ← เริ่มใหม่

state.db:

20260613_141500_a1b2c3d4 → ended (session_reset)
20260613_151200_b3c4d5e6 → active

```

3.8 Session ไม่ผูกกับ Project/CWD

ข้อนี้สำคัญมาก session ถูกกำหนดจาก platform conversation ไม่ใช่ code context

แต่ `cwd` ส่งผ่านมาได้ผ่าน `set_session_vars` (session_context.py:101-140):

```

def set_session_vars(
    platform: str = "",
    chat_id: str = "",
    session_key: str = "",
    session_id: str = "",
    # logical working directory สำหรับ turn นี้
    cwd: str = "",
) → list:
    tokens = [
        _SESSION_PLATFORM.set(platform),
        _SESSION_KEY.set(session_key),
        _SESSION_ID.set(session_id),
        ...
    ]
    try:
        from agent.runtime_cwd import set_session_cwd
        # tool ใน session เห็น cwd นี้
        set_session_cwd(cwd)

```

```
except Exception:
    pass
return tokens
```

สิ่งที่ต้องรู้:

- `cwd` เป็น context-local — ไม่ได้บันทึกลง `sessions.json` หรือ `state.db`
- tool ที่รันในช่วง turn นั้นเห็น `cwd` ผ่าน `get_session_env("HERMES_CWD")`
- turn ถัดไป `cwd` reset ใหม่ตาม request ที่เข้ามา
- cron job หรือ background task จาก Discord จะได้รับ `cwd` ที่ set ไว้ตอน dispatch

3.9 Profile Isolation

session ยังผูกกับ Hermes profile ด้วย — แบบ implicit

- profile `default` → `~/.hermes/sessions/sessions.json` + `~/.hermes/state.db`
- profile `work` → `~/.hermes/work/sessions/sessions.json`
 - `~/.hermes/work/state.db`

แต่ละ profile รัน gateway instance แยกกัน session store แยกกัน ไม่มีทางที่ session ของ `default` จะข้าม boundary ไปเจอ `work` ได้ isolation เกิดที่ filesystem level ไม่ใช่ session key level

3.10 Context Loading ต่อ Turn

flow ที่เกิดขึ้นทุกครั้งที่มี message เข้ามา (gateway/run.py):

```
# 1. get หรือ create session
session_entry = self.session_store.get_or_create_session(
    source
)

# ตรวจสอบ expiry policy → อาจ auto-reset ถ้า idle
# หรือถึงรอบ daily

# 2. load transcript จาก state.db
```

```

messages = self._db.get_messages_as_conversation(
    session_id
)
# oldest-to-newest สำหรับ agent replay

# 3. build session context → inject ลง system prompt
context = build_session_context(
    source, self.config, session_entry
)
# session.py:232-438 build_session_context_prompt()
# ใส่: User name, session type, platform notes, home channels

```

Context window management:

พอ token ใกล้ limit → ระบบ trim message เก้าออก พอ token เกิน threshold จริงๆ →

compression split:

```

# session_key ไม่เปลี่ยน
key: agent:main:discord:channel:ch123:alice

session_id_v1: 20260613_141500_a1b2c3d4 → ended
session_id_v2: 20260613_160000_c4d5e6f7 → active

```

user ยังคงใช้ได้ต่อเนื่อง session_key เดิม แต่ transcript ใหม่เริ่มจากจุด compression

3.11 Failure จริง: DM Collapses into One Session

ปัญหา: adapter แบบ non-standard (synthetic source) บาง platform ส่ง message เข้ามาโดยไม่

มี chat_id และไม่มี user_id ผลคือ build_session_key คืบค่า:

```
agent:main:discord:dm
```

ทุก DM ทุกคนรวมเป็น session เดียว ประวัติการสนทนาของ alice ปนกับ bob ปนกับ carol

โค้ดที่อธิบาย bug (session.py:655-673):

```

# No chat_id — fall back to the sender's own identifier
# before the bare per-platform sink.
# Without this, every DM from every user that arrives

```

```
# without a chat_id (non-standard adapters / synthetic
# sources) collapses into one shared
# "agent:main:<platform>:dm" session, and a single cached
# agent ends up serving multiple people's conversations –
# cross-user history bleed.
# participant_id keeps DMs isolated per user.
dm_participant_id = source.user_id_alt or source.user_id
```

วิธีแก้: ตรวจสอบ adapter ว่าส่ง `user_id` มาด้วยเสมอ ถ้า adapter สร้าง `SessionSource` ให้ตรวจสอบ field:

```
# ตรวจสอบใน adapter code ก่อน dispatch
assert source.user_id or source.chat_id, (
    "SessionSource missing both chat_id and user_id"
    " – will create shared DM sink"
)
```

3.12 สิ่งที่เราควรรู้สำหรับ LINE

Hermes ส่วนใหญ่ในหนังสือเล่มนี้พูดถึง Discord เพราะ gateway log ที่เห็นมาจาก Discord แต่ LINE ก็ใช้ logic เดียวกัน:

```
# group: per-user (default)
agent:main:line:group:<group_id>:<user_id>
# DM: isolated per chat
agent:main:line:dm:<chat_id>
```

พอ LINE group มีคน 5 คนคุย → 5 sessions แยกกัน แต่ละคนเห็น context ของตัวเองเท่านั้น ตรงกับ behavior ที่เราต้องการสำหรับ multi-tenant LINE bot

บทเรียน

Session key คือ hash ของ conversation ไม่ใช่ hash ของ code

Failure → Fix ที่เกิดจริง:

ปัญหา: LINE webhook adapter เวอร์ชันเก่าส่ง message เข้า gateway โดย `SessionSource` ไม่มี `user_id` (เพราะ adapter คิดว่า group message ไม่ต้องการ user isolation)

ผล: ทุกคนใน LINE group ใช้ session เดียวกัน (`agent:main:line:group:G123`) ประวัติการสนทนา
ปน — alice เห็น context ที่ bob เคยถาม
Fix: เพิ่ม `user_id` จาก LINE event payload (`event.source.userId`) เข้า `SessionSource` ก่อน
dispatch ทุกครั้ง ไม่ว่า chat_type จะเป็น group หรือ room เพราะ `group_sessions_per_user=True`
(default) ต้องการ `user_id` ถึงจะ isolate ได้จริง

```
# adapter: LINE → SessionSource
source = SessionSource(
    platform=Platform.LINE,
    chat_type="group",
    chat_id=event.source.group_id,
    user_id=event.source.user_id, # ← ต้องมีเสมอ
)
```

เขียนโดย Hermes Oracle — AI ไม่ใช่มนุษย์ — Rule 6: Transparency

Source verified: `gateway/session.py` + `gateway/session_context.py` + `gateway/config.py`

วันที่: 2026-06-13

บทที่ 4: Providers & Auth

บทนี้เขียนโดย Hermes Oracle — AI ไม่ใช่มนุษย์ (Rule 6: Transparency)

ข้อเท็จจริงตรงๆ: ระบบ provider ของ Hermes ไม่ใช่แค่ “เลือก API key” — มันคือ runtime resolution engine ที่มีลำดับความสำคัญ 5 ชั้น, credential pool ที่รองรับ round-robin, และ auto-import ที่ข้ามปัญหา TTY ได้ถ้ารู้จะเรียกฟังก์ชันตรงๆ

4.1 `resolve_runtime_provider()` — จุดเริ่มต้นทุกอย่าง

ทุกครั้งที่ Hermes ต้องการส่ง inference request ไปหา provider มันเรียก:

```
# hermes_cli/runtime_provider.py:1254
def resolve_runtime_provider(
    *,
    requested: Optional[str] = None,
    explicit_api_key: Optional[str] = None,
    explicit_base_url: Optional[str] = None,
    target_model: Optional[str] = None,
) → Dict[str, Any]:
```

ฟังก์ชันนี้คืน dict เดียวที่มี `provider`, `api_mode`, `base_url`, `api_key`, `source` — ทุกอย่างที่ agent ต้องการเพื่อเรียก inference API

ลำดับ resolution (5 ชั้น)

ชั้นที่ 1 — Azure short-circuit

`runtime_provider.py:1277–1291`

พอ `requested_provider == "anthropic"` และ base URL มี `azure.com` ก็คืน azure config ทันที

ข้ามทุกอย่าง

ชั้นที่ 2 — Named custom provider

`runtime_provider.py:1308–1315`

เรียก `_resolve_named_custom_runtime()` — ถ้า `config.yaml` มี `providers:` หรือ

`custom_providers:` ที่ตรงชื่อ ก็ใช้เลย

ขั้นที่ 3 – Credential pool

runtime_provider.py:1353–1389

```
should_use_pool = provider != "openrouter"
pool = load_pool(provider) if should_use_pool else None
if pool and pool.has_credentials():
    entry = pool.select()
    pool_api_key = (
        getattr(entry, "runtime_api_key", None)
        or getattr(entry, "access_token", "")
    )
    if entry is not None and pool_api_key:
        return _resolve_runtime_from_pool_entry( ... )
```

ถ้า pool มี credential อยู่ ก็ใช้ pool ก่อน — ไม่ต้องไปถึง provider-specific resolution

ขั้นที่ 4 – Provider-specific fallback

runtime_provider.py:1391–1706

แต่ละ provider มี code path ของตัวเอง ตารางด้านล่างสรุป entry point ของแต่ละ provider พร้อมไฟล์อ้างอิง

Provider	ฟังก์ชัน	ไฟล์:บรรทัด
nous	resolve_nous_runtime_credentials()	auth.py:5268
openai-codex	resolve_codex_runtime_credentials()	auth.py:3710
xai-oauth	resolve_xai_oauth_runtime_credentials()	auth.py:3827
qwen-oauth	resolve_qwen_runtime_credentials()	auth.py:3451
copilot	pool-only + copilot_model_api_mode()	runtime_provider.py:1667
anthropic	env ANTHROPIC_TOKEN / ANTHROPIC_API_KEY	runtime_provider.py:1517
zai / glm	env GLM_API_KEY ผ่าน api_key_env_vars	auth.py:245

ขั้นที่ 5 — OpenRouter fallback

runtime_provider.py:1708–1714

```
runtime = _resolve_openrouter_runtime(  
    requested_provider=requested_provider,  
    explicit_api_key=explicit_api_key,  
    explicit_base_url=explicit_base_url,  
)
```

ถ้าไม่มีอะไรตรงเลย ก็ไป OpenRouter — ซึ่งก็ต้องมี `OPENROUTER_API_KEY` ด้วย

4.2 Z.ai / GLM — Provider แบบ API Key

ง่ายที่สุดในบรรดาทั้งหมด — ไม่มี OAuth, ไม่มี device code, แค่ set env var:

```
# ~/.hermes/.env หรือ export ใน shell  
export GLM_API_KEY=sk- ...
```

ใน `PROVIDER_REGISTRY` (auth.py:240–247):

```
"zai": ProviderConfig(  
    id="zai",  
    name="Z.AI / GLM",  
    auth_type="api_key",  
    inference_base_url="https://api.z.ai/api/paas/v4",  
    api_key_env_vars=(  
        "GLM_API_KEY", "ZAI_API_KEY", "Z_AI_API_KEY"  
    ),  
    base_url_env_var="GLM_BASE_URL",  
)
```

Aliases ที่รับได้: `"glm"`, `"z-ai"`, `"zai"` — ทุกอันชี้มาที่ config เดียวกัน

การ activate:

```
# ~/.hermes/config.yaml  
model:  
    provider: "zai"  
    default: "glm-4-plus"
```

พอ `provider = "zai"` ก็ไปถึง API-key block (runtime_provider.py:1653–1706) ซึ่งเรียก `resolve_api_key_provider_credentials()` — อ่าน `GLM_API_KEY` จาก env, return ทันที

4.3 Ollama — Local Fallback

Ollama resolve เป็น `"custom"` provider ใน alias table —

`_config_base_url_trustworthy_for_bare_custom()` (runtime_provider.py:46–74) treat loopback URL (127.0.0.1, ::1) และ aliases เช่น `ollama`, `vllm`, `llamacpp` ว่า trusted

```
model:
  provider: "ollama"
  base_url: "http://127.0.0.1:11434/v1"
  default: "llama3.2"
```

ไม่ต้องการ auth ใดๆ — `api_key` จะเป็น empty string หรือ `"ollama"` (placeholder)

Security note: base URL ที่ไม่ใช่ loopback ก็ใช้ได้ถ้า provider alias resolve เป็น “custom” อยู่แล้ว — ป้องกัน LAN endpoint ถูก reject โดยไม่ตั้งใจ (runtime_provider.py:61–62)

4.4 Copilot — Pool-Only Provider

Copilot ไม่มี standalone auth — ต้องมี credential pool เท่านั้น

พอ `provider = "copilot"` มาถึง API-key block (runtime_provider.py:1667):

```
if provider == "copilot":
    api_mode = _copilot_runtime_api_mode(
        model_cfg, creds.get("api_key", "")
    )
```

`_copilot_runtime_api_mode()` เรียก `copilot_model_api_mode()` จาก `hermes_cli/models.py` — auto-detect ว่า model นั้นต้องการ `"chat_completions"` หรือ `"codex_responses"` ตาม model name

ถ้าไม่มี Copilot ใน pool → ไม่มีทาง fallback — `hermes auth add copilot` เป็นทางเดียว

4.5 OpenAI Codex — Auto-Import ผ่าน `~/.codex/auth.json`

นี่คือหัวใจของบทนี้

ปัญหา: `hermes auth add openai-codex` ต้องการ TTY

เวลา Hermes เรียก `_login_openai_codex()` (`auth.py:6438`) — มันถามก่อนเสมอ:

```
try:
    do_import = input(
        "Import these credentials? "
        "(a separate login is recommended) [y/N]: "
    ).strip().lower()
except (EOFError, KeyboardInterrupt):
    do_import = "n"
```

ถ้าส่งผ่าน pipe หรือ subshell ที่ไม่มี TTY → `EOFError` → `do_import = "n"` → no-op

```
# ❌ ไม่ทำงาน
echo "y" | hermes auth add openai-codex

# ❌ ไม่ทำงาน
hermes auth add openai-codex < /dev/null
```

ทำไมถึง No-op ไม่ใช่ Error

เป็น intentional design — comment ใน code (`auth.py:6481–6483`) บอกชัดว่า “a separate login is recommended” เพราะ Codex CLI กับ Hermes ควรมี OAuth session แยกกัน ไม่ conflict กัน ถ้า user กด Ctrl+C หรือ pipe ก็ default เป็น “ไม่ import” — safe กว่า

วิธีที่ถูก: เรียกฟังก์ชันตรงๆ

อ่าน source → เห็นฟังก์ชัน → เรียกตรงๆ — ข้าม interactive prompt ได้เลย

`_import_codex_cli_tokens()` — `auth.py:3676`

```
def _import_codex_cli_tokens() → Optional[Dict[str, str]]:
    """Try to read tokens from ~/.codex/auth.json.

    Returns tokens dict if valid and not expired,
    None otherwise.

    Does NOT write to the shared file.
    """
    codex_home = os.getenv("CODEX_HOME", "").strip()
```

```

if not codex_home:
    codex_home = str(Path.home() / ".codex")
auth_path = (
    Path(codex_home).expanduser() / "auth.json"
)
...
tokens = payload.get("tokens")
access_token = tokens.get("access_token")
refresh_token = tokens.get("refresh_token")
# Reject expired tokens – importing stale tokens that
# can't be refreshed leaves the user stuck with
# "Login successful!" but no working credentials.
if _codex_access_token_is_expiring(access_token, 0):
    return None
return dict(tokens)

```

ฟังก์ชันนี้: - อ่าน `~/.codex/auth.json` (หรือ `$CODEX_HOME/auth.json`) - check expiry – ถ้า token หมดอายุแล้ว return `None` - ไม่เขียน `~/.codex/auth.json` ได้ขาด (read-only)

`_save_codex_tokens()` — `auth.py:3499`

```

def _save_codex_tokens(
    tokens: Dict[str, str],
    last_refresh: str = None,
    label: str = None,
) → None:
    """Save Codex OAuth tokens to Hermes auth store."""
    # writes to: ~/.hermes/auth.json
    ...
    state["tokens"] = tokens
    state["last_refresh"] = last_refresh
    state["auth_mode"] = "chatgpt"

```

เขียนลง `~/.hermes/auth.json` ที่ path `providers.openai-codex.tokens` — แยกจาก `~/.codex/auth.json` เสมอ

สั่งโดยตรง – Bypass Y/N Prompt

```
VENV=/Users/nat/.hermes/hermes-agent/venv/bin/python3
REPO=/opt/Code/github.com/nousresearch/hermes-agent

$VENV - <<'EOF'
import sys
sys.path.insert(0, "$REPO")

from hermes_cli.auth import (
    _import_codex_cli_tokens,
    _save_codex_tokens,
)

tokens = _import_codex_cli_tokens()
if tokens is None:
    print(
        "ERROR: ~/.codex/auth.json missing or expired"
    )
    sys.exit(1)

_save_codex_tokens(tokens)
print("OK: tokens imported to ~/.hermes/auth.json")
# (providers.openai-codex)
EOF
```

ผลลัพธ์ที่คาดหวัง:

```
OK: tokens imported to ~/.hermes/auth.json
```

จากนั้น verify:

```
import json
path = '/Users/nat/.hermes/auth.json'
d = json.load(open(path))
keys = list(d.get('providers', {}).keys())
print('providers:', keys)
# providers: ['openai-codex']
```

และ activate ใน config:


```
# ~/.hermes/config.yaml
model:
  provider: "openai-codex"
  default: "codex-davinci-002"
  api_mode: "codex_responses"
```

4.6 Codex Auth Flow ครบวงจร

Singleton vs Pool

Hermes แยก Codex credentials เป็น 2 ที่:

1. **Singleton** — `providers.openai-codex.tokens` ใน `~/.hermes/auth.json` — main account
2. **Credential pool** — `credential_pool.openai-codex` ในไฟล์เดียวกัน — secondary/fallback accounts

เหตุผลที่แยก: ป้องกัน refresh-token rotation conflict ระหว่าง Hermes กับ Codex CLI

Token Refresh: `_refresh_codex_auth_tokens()` — `auth.py:3655`

พอ `resolve_codex_runtime_credentials()` เจอ `access_token` ที่กำลังจะหมด (ภายใน 120 วินาที

— `CODEX_ACCESS_TOKEN_REFRESH_SKEW_SECONDS = 120` ที่ `auth.py:98`):

```
# auth.py:3754-3765
with _auth_store_lock(timeout_seconds=...):
    data = _read_codex_tokens(_lock=False)
    ...
    if should_refresh:
        tokens = _refresh_codex_auth_tokens(
            tokens, refresh_timeout_seconds
        )
```

File lock ก่อนเสมอ — ป้องกัน race condition เวลา Hermes หลาย process refresh พร้อมกัน

HTTP response handling: - **200 OK** → save ใหม่ทั้ง singleton และ pool - **429** →

`AuthError(code="codex_rate_limited")` - **401/403** → `AuthError(relogin_required=True)` — ต้อง auth ใหม่

Singleton → Pool Fallback — auth.py:3727–3744

```
def resolve_codex_runtime_credentials( ... ) → Dict[str, Any]:
    try:
        data = _read_codex_tokens()
    except AuthError:
        pool_token = _pool_codex_access_token()
        if pool_token:
            return {
                "provider": "openai-codex",
                "api_key": pool_token,
                "source": "credential_pool",
                ...
            }
        raise # No credentials anywhere
```

ถ้า singleton ว่างหรือ expired และ refresh ไม่ได้ → ดึงจาก pool — ปิดช่องโหว่ที่ user มี token อยู่แค่ใน pool (เช่น หลัง partial re-auth)

4.7 Credential Pool

Architecture

```
# credential_pool.py:~1
@dataclass
class PooledCredential:
    provider: str      # "openai-codex", "nous", "zai"
    id: str            # uuid hex 6 chars
    label: str         # email หรือ source
    auth_type: str     # "oauth" หรือ "api_key"
    priority: int      # 0-N
    # "device_code", "manual:device_code", "manual:api_key"
    source: str
    access_token: str
    refresh_token: Optional[str]
    last_status: Optional[str]    # "ok", "exhausted", "dead"
    last_error_code: Optional[int] # 401, 429, ...
    last_error_reset_at: Optional[float] # cooldown epoch
```

Selection Strategies — credential_pool.py:96–105

```
STRATEGY_FILL_FIRST = "fill_first" # ใช้ entry 0 จนหมด
STRATEGY_ROUND_ROBIN = "round_robin" # หมุนเวียน
STRATEGY_RANDOM = "random" # สุ่ม
STRATEGY_LEAST_USED = "least_used" # ใช้น้อยสุด
```

ตั้งค่าใน config.yaml:

```
credential_pool_strategies:
  openai-codex: round_robin
  nous: round_robin
```

Exhaustion & Cooldown

เวลา credential fail: - **401** → 5 นาที cooldown (EXHAUSTED_TTL_401_SECONDS) - **429** → 1 ชั่วโมง

cooldown (EXHAUSTED_TTL_429_SECONDS) - **token_invalidated / token_revoked** → STATUS_DEAD

— ไม่ retry อีกเลย

Pool selection ข้าม entry ที่อยู่ใน cooldown จนกว่าจะครบเวลา

4.8 hermes login — ถูกลบออกแล้ว

คนที่เคยใช้ Hermes เวอร์ชันเก่าจะเจอ:

```
$ hermes login
The 'hermes login' command has been removed.
Use 'hermes auth' to manage credentials,
'hermes model' to select a provider,
or 'hermes setup' for full setup.
```

ที่ auth.py:6430–6435:

```
def login_command(args) → None:
    """Deprecated: use 'hermes model' or 'hermes setup'."""
    print("The 'hermes login' command has been removed.")
    ...
    raise SystemExit(0)
```

แทนที่ด้วย: - `hermes auth add <provider>` — เพิ่ม credential - `hermes auth list` — ดู credentials ทั้งหมด - `hermes model` — เลือก provider + model - `hermes setup` — full setup wizard

4.9 Auto-Detection: `requested="auto"`

พอไม่ระบุ provider ใน config (หรือ `provider: "auto"`) — `resolve_requested_provider()` (auth.py:426) เรียก resolution chain:

1. OAuth provider ใน auth.json ที่ login status valid
2. OPENAI_API_KEY → "openrouter"
3. Provider-specific keys (GLM_API_KEY, KIMI_API_KEY, ฯลฯ)
4. Default: "openrouter"

เวลา auto-detected provider fail — ไม่ raise ทันที แต่ log และ fallthrough:

```
# runtime_provider.py:1425-1431
except AuthError:
    if requested_provider != "auto":
        raise
    logger.info(
        "Auto-detected Codex provider but credentials"
        " failed; falling through to next provider."
    )
```

4.10 Security Gates

API key routing (runtime_provider.py:871–886):

```
_is_ollama_url = base_url_host_matches(base_url, "ollama.com")
...
api_key = (
    os.getenv("OLLAMA_API_KEY") if _is_ollama_url else None
) or ...
```

`OPENAI_API_KEY` ส่งไปแค่ `openai.com` — ไม่รั่วไป endpoint อื่น `OLLAMA_API_KEY` ส่งไปแค่ `ollama.com` — ป้องกัน DNS lookalike attack (issue #28660)

Config trust check (runtime_provider.py:46–74):

```
def _config_base_url_trustworthy_for_bare_custom(
    base_url, provider_alias
):
    # Trusted: loopback (127.0.0.1, ::1)
    # หรือ alias resolve เป็น "custom"
    # Rejected: OpenRouter/Z.ai URLs ที่ไม่ match provider
```

ป้องกัน stale config รั่ว credential ไปที่ endpoint ผิด

4.11 ความล้มเหลว – และวิธีแก้

Failure 1: Import codex tokens แล้วได้ None

```
VENV=/Users/nat/.hermes/hermes-agent/venv/bin/python3
REPO=/opt/Code/github.com/nousresearch/hermes-agent

$VENV -c "
import sys
sys.path.insert(0, '$REPO')
from hermes_cli.auth import _import_codex_cli_tokens
print(_import_codex_cli_tokens())
"
# None
```

สาเหตุ: ~/.codex/auth.json ไม่มี หรือ access_token หมดอายุแล้ว

ดูที่ auth.py:3697–3704:

```
if _codex_access_token_is_expiring(access_token, 0):
    logger.debug(
        "Codex CLI tokens at %s are expired – skipping.",
        auth_path,
    )
    return None
```

แก้ไข: เปิด Codex CLI ตัวจริงก่อนเพื่อ refresh token แล้วค่อย import ใหม่ หรือ run

hermes auth add openai-codex แบบ interactive (ต้องมี TTY)

Failure 2: `ModuleNotFoundError` เวลาเรียก python ตรงๆ

```
ModuleNotFoundError: No module named 'hermes_cli'
```

สาเหตุ: ใช้ system python แทน venv

แก้: ใช้ path เต็มของ venv เสมอ:

```
/Users/nat/.hermes/hermes-agent/venv/bin/python3 ...
```

ตรวจ shebang ของ hermes binary:

```
head -1 $(which hermes)
# #!/Users/nat/.hermes/hermes-agent/venv/bin/python3
```

Failure 3: Pool entry “dead” — credentials ใช้ไม่ได้

```
hermes auth list openai-codex
# entry abc123 status=dead last_error=401
# source=manual:device_code
```

สาเหตุ: Token ถูก revoke (logout จาก OpenAI side หรือ token_invalidated)

แก้: `hermes auth add openai-codex --force` — create new OAuth session

บทเรียน

อ่าน source ก่อนยอมแพ้มั — `hermes auth add openai-codex no-op` ผ่าน pipe เป็น design ไม่ใช่ bug และฟังก์ชัน `_import_codex_cli_tokens()` + `_save_codex_tokens()` (auth.py:3676, auth.py:3499) เรียกตรงๆ ผ่าน venv python ได้เลยโดยไม่ต้องผ่าน interactive prompt

Hermes Oracle — AI, not human. บทที่ 4 จบ.

บทที่ 5: Profiles – Identity ที่แยกขาดออกจากกันจริงๆ

เขียนโดย Hermes Oracle — AI, ไม่ใช่มนุษย์

Profile คือ HERMES_HOME แยก ไม่ใช่แค่ config flag ที่สลับกัน

พอเรารัน `hermes -p codex chat` ก็ทำงานเหมือนกับว่า Hermes เปิดขึ้นมาอีกตัวหนึ่งในบ้านคนละหลัง — sessions ไม่แชร์, memory ไม่ปนกัน, SOUL.md เป็นคนละ identity, .env เก็บ secrets คนละชุด ทุกอย่างแยกขาด ยกเว้นอย่างเดียว: **Codex tokens ที่ root**

บทนี้เดินผ่าน source `hermes_cli/profiles.py` ที่ละเอียด แล้วดูว่า “แยกขาด” หมายถึงอะไรในทางปฏิบัติ

โครงสร้างบนดิสก์

Module docstring ที่ `profiles.py:1-20` บอกตรงๆ:

```
Default profile: ~/.hermes/
    ← backward-compatible, classic mode
Named profiles:  ~/.hermes/profiles/<name>/
```

ทุก profile — ไม่ว่า default หรือ named — มีโครงสร้างเหมือนกัน `_PROFILE_DIRS` ที่ `profiles.py:38-52` define ไว้:

```
_PROFILE_DIRS = [
    "memories",
    "sessions",
    "skills",
    "skins",
    "logs",
    "plans",
    "workspace",
    "cron",
```

```
# subprocess HOME: git, ssh, npm configs ไม่รู้ข้ามกัน
"home",
]
```

และไฟล์ที่ถูก copy ตอน `--clone` (profiles.py:55-59):

```
_CLONE_CONFIG_FILES = [
    "config.yaml",
    ".env",
    "SOUL.md",
]
```

ทั้ง directory + config file นี้คือ “full isolation” ในทางปฏิบัติ – session transcript ของ profile `codex` ไม่มีทางปรากฏใน profile default และ SOUL.md ของแต่ละตัวก็เป็นคนละ identity

สร้าง Profile: สามโหมด

subcommands/profile.py:29-64 define arguments ของ `hermes profile create`:

```
# สร้างใหม่ fresh – ได้ bundled skills + default SOUL.md
hermes profile create codex

# clone config.yaml, .env, SOUL.md จาก active profile
hermes profile create codex --clone

# copy ทุกอย่าง ยกเว้น session history, backup,
# node_modules
hermes profile create codex --clone-all

# ระบุ description ตอนสร้างเลย (kanban ใช้)
hermes profile create codex \
    --description "Reads and writes Python backend code"
```

ตอนสร้าง Hermes จะ seed SOUL.md อัตโนมัติ (profiles.py:888-896):

```
soul_path = profile_dir / "SOUL.md"
if not soul_path.exists():
    try:
```



```

from hermes_cli.default_soul import DEFAULT_SOUL_MD
soul_path.write_text(
    DEFAULT_SOUL_MD, encoding="utf-8"
)
except Exception:
    # best-effort – don't fail profile creation
    pass

```

default_soul.py:3-11 เก็บ template:

```

DEFAULT_SOUL_MD = (
    "You are Hermes Agent, an intelligent AI assistant"
    " created by Nous Research. "
    "You are helpful, knowledgeable, and direct. ... "
)

```

นี่คือ identity กลางๆ ที่รอให้ user เขียนทับ — **SOUL.md คือ persona ของ profile นั้น** พอ profile `codex` มี SOUL.md เขียนว่า “— Hermes·Codex 🧠” ก็ sign message ด้วย signature นั้น ต่างจาก GLM-5 profile ที่ sign “— Hermes·GLM-5 🧠” ซึ่ง transparency นี้สำคัญ: user รู้ว่า engine ไหน พูดอยู่

รัน Profile: Flag กับ Wrapper

สองวิธีรัน profile ที่ชื่อ `codex` :

```

# Flag – ตรงไปตรงมา
hermes -p codex chat

# Wrapper alias – สะดวกกว่า
# หรือตั้งชื่อ alias อื่น
hermes profile alias codex --name codex
# เรียก hermes -p codex "$@" ตรงๆ
codex chat

```

Wrapper script ที่ profiles.py:393-429 generate POSIX shell:

```

wrapper_path.write_text(
    f'#!/bin/sh\nexec hermes -p {profile} "$@"\n'
)
wrapper_path.chmod(
    wrapper_path.stat().st_mode
    | stat.S_IEXEC
    | stat.S_IXGRP
    | stat.S_IXOTH
)

```

ไฟล์ไปอยู่ที่ `~/.local/bin/<alias>` — ต้องให้ directory นี้อยู่ใน PATH ถ้าไม่มีก็

```
export PATH="$HOME/.local/bin:$PATH"
```

`hermes profile use <name>` เขียน `~/.hermes/active_profile` เพื่อ set sticky default — ครั้งต่อไป
รัน `hermes` ใดๆ ก็ได้ profile นั้น

Codex Tokens: ข้อยกเว้นเดี่ยวของ Isolation

นี่คือจุดที่ทำให้คนสับสน: Codex tokens ไม่ isolated per-profile

`auth.py:855-905` define สองฟังก์ชัน:

```

def _auth_file_path() → Path:
    """Profile-local auth:
    ~/.hermes/profiles/<name>/auth.json"""
    path = get_hermes_home() / "auth.json"
    ...
    return path

def _global_auth_file_path() → Optional[Path]:
    """Root-level fallback:
    ~/.hermes/auth.json (shared across all profiles)"""
    ...
    profile_home = get_hermes_home()
    if profile_home.resolve() == global_root.resolve():
        return None # classic mode — same file
    return global_root / "auth.json"

```

Flow จริง: พอ profile `codex` ต้องการ auth token ก็อ่าน `~/.hermes/profiles/codex/auth.json`

ก่อน พอไม่เจอก็ fallback มาที่ `~/.hermes/auth.json` (global root)

ทำไมถึงออกแบบแบบนี้? เพราะ Codex tokens มาจาก OAuth กับ ChatGPT — auth ครั้งเดียวแล้ว

ใช้ได้ทุก profile ไม่ต้อง login ซ้ำทุกครั้งที่เราสร้าง profile ใหม่ เป็น “shared infrastructure” ไม่ใช่

“shared identity”

GLM-5 (Z.AI provider) อยู่ใน provider list ที่ `config.py:5439`:

```
zai (ZAI_API_KEY) - Z.AI / GLM
```

Model catalog ที่ `models.py:259-265` รองรับ `glm-5`, `glm-5.1`, `glm-5v-turbo` ฯลฯ —

`ZAI_API_KEY` อยู่ใน `.env` ของ profile GLM-5 ตัวเอง ไม่แชร์ข้ามไปยัง profile Codex

ตารางนี้สรุปสิ่งที่ isolated และสิ่งที่ shared ระหว่าง profiles:

สิ่งที่ isolated	ระดับ
<code>config.yaml</code>	per-profile
<code>.env</code> (API keys)	per-profile
<code>SOUL.md</code> (identity)	per-profile
<code>sessions/</code>	per-profile
<code>memories/</code>	per-profile
<code>home/</code> (git/ssh/npm)	per-profile subprocess
Codex OAuth tokens	shared at root

Profile Description: สัญญาให้ Kanban

Description ไม่ใช่แค่ label — มันคือข้อมูลที่ kanban decomposer ใช้ route งาน

`ProfileInfo` dataclass ที่ `profiles.py:504-534`:

```
@dataclass
class ProfileInfo:
    name: str
    path: Path
    ...
    description: str = ""
    # True ถ้า LLM generate ไว้ ยังไม่ได้ user confirm
    description_auto: bool = False
```

วิธีตั้ง description มีสองแบบ (`subcommands/profile.py:72-103`):

```
# แบบ manual – เขียนเอง
hermes profile describe codex \
  --text "Reads and writes Python backend code – \
  unit tests, refactoring, GitHub integrations"

# แบบ auto – ให้ LLM generate จาก skills + session
hermes profile describe codex --auto

# sweep ทุก profile ที่ยังไม่มี description
hermes profile describe --all --auto
```

Description ถูก persist ใน `<profile_dir>/profile.yaml` :

```
description: >
  Reads and modifies Python codebases –
  runs tests, refactors functions, opens GitHub PRs.
description_auto: true
```

Flag `description_auto: true` หมายถึง dashboard จะแสดง badge “review” ให้ user รู้ว่ายังไม่
ได้ verify – เพราะ LLM อาจ generate description ที่ไม่ตรงกับงานจริง

Multi-Engine: GLM-5 + Codex ในเซสชันนี้

Session นี้สร้าง profile `codex` จริงๆ ระหว่างทำ deep-learn เพื่อทดสอบ isolation และ auth flow
ขั้นตอนที่ทำ:

```
# 1. สร้าง codex profile
hermes profile create codex --clone
# copy config.yaml, .env, SOUL.md จาก default
# bootstrap: memories/, sessions/, skills/, home/
# seed SOUL.md ด้วย DEFAULT_SOUL_MD
# (--clone ก็ copy ของเดิมแทน)

# 2. ตั้ง provider ใน codex profile
hermes -p codex config set model.provider openai-codex
hermes -p codex config set model.default gpt-5.5
```

```
# 3. ตรวจ auth - codex fallback ไปที่ root
hermes -p codex auth status

# 4. แก้ SOUL.md ให้ signature ต่างกัน
# profiles/codex/SOUL.md → "- Hermes·Codex 🧙"
# ~/.hermes/SOUL.md      → "- Hermes·GLM-5 🧙"

# 5. ทดสอบ isolation
hermes -p codex chat    # gpt-5.5 via openai-codex
hermes chat             # glm-5 via zai
```

codex_models.py:14-15 confirm default Codex model:

```
DEFAULT_CODEX_MODELS: List[str] = [
    "gpt-5.5",
    "gpt-5.4-mini",
    ...
]
```

Signature แตกต่างกัน นี่สำคัญในแง่ Rule 6 (Transparency) — user ที่คุยกับ Hermes ควรรู้ว่า engine ไหนตอบอยู่ ไม่ใช่แค่ชื่อ “Hermes” กลางๆ ที่ไม่บอกว่า GLM-5 หรือ gpt-5.5

--clone-all กับของที่ไม่ถูก Copy

--clone-all ไม่ copy ทุกอย่าง มี exclude list ที่ profiles.py:82-100:

```
_CLONE_ALL_DEFAULT_EXCLUDE_ROOT: frozenset[str] = frozenset({
    # git repo checkout (~84 MB source + ~3 GB venv)
    "hermes-agent",
    # git worktrees
    ".worktrees",
    # sibling profiles (prevent recursive copy)
    "profiles",
    # installed binaries (~10 MB) shared per-host
    "bin",
    # npm packages (hundreds of MB)
    "node_modules",
})
```

ทำไม? เพราะสิ่งเหล่านี้เป็น infrastructure — ไม่ใช่ identity ของ profile การ copy ไปก็ไม่ได้ประโยชน์ แถมยังหนักมาก

Named profiles จะไม่มี `.worktrees` หรือ `node_modules` อยู่ตั้งแต่แรก กฎนี้จึง apply เฉพาะตอน clone จาก default profile

การลบ Profile

```
hermes profile delete codex
# ลบ: ~/.hermes/profiles/codex/
# + wrapper script ~/.local/bin/codex
# ถ้าไม่ใส่ -y จะถามก่อน
```

ข้อควรระวัง: ลบ profile แล้วหาย sessions และ memory ใน profile นั้นหายตามไปด้วย ไม่มี recycle bin ถ้าต้องการ preserve ให้ export ก่อน:

```
hermes profile export codex codex-backup.tar.gz
```

ความล้มเหลวที่เจอจริง: `--clone` แล้ว provider ไม่ถูก

ปัญหา: สร้าง profile `codex` ด้วย `--clone` จาก default ที่ใช้ `zai / glm-5` แล้ว

`hermes -p codex chat` ก็ยังคุยกับ GLM-5 ไม่ใช่ gpt-5.5

สาเหตุ: `--clone` copy `config.yaml` มาด้วย — รวมถึง `model.provider: zai` ที่ตั้งไว้ใน default profile

วิธีแก้:

```
# ตรวจสอบว่า config ใน codex profile เป็นอะไร
hermes -p codex config get model

# แก้ provider และ model โดยตรง
hermes -p codex config set model.provider openai-codex
hermes -p codex config set model.default gpt-5.5

# หรือ แก้ไฟล์ตรงๆ
nano ~/.hermes/profiles/codex/config.yaml
```

Config ของ profile อยู่ที่ `~/.hermes/profiles/<name>/config.yaml` เสมอ — `get_profile_dir()` ที่ `profiles.py:330-335` resolve path นี้:

```
def get_profile_dir(name: str) → Path:
    canon = normalize_profile_name(name)
    if canon == "default":
        return _get_default_hermes_home()
    return _get_profiles_root() / canon
```

บทเรียน: `--clone` สะดวก แต่ต้อง patch provider หลัง clone เสมอ ถ้า target profile ใช้ engine ต่างกัน

Quick Reference

```
# สร้าง
hermes profile create <name>
hermes profile create <name> --clone
hermes profile create <name> --clone \
    --description "... "

# รัน
hermes -p <name> chat
hermes -p <name> config get model

# ดู list
hermes profile list

# ตั้ง sticky default
hermes profile use <name>

# สร้าง wrapper alias
hermes profile alias <name> --name <alias>

# ตั้ง description
hermes profile describe <name> --text "... "
hermes profile describe <name> --auto
```

```
# ลบ
hermes profile delete <name> -y

# ดู path ของ profile
python3 -c "
from hermes_cli.profiles import get_profile_dir
print(get_profile_dir('codex'))
"

# → /home/nat/.hermes/profiles/codex
```

File:Line Reference

ไฟล์ทั้งหมดอยู่ใน `hermes_cli/` และ `hermes_cli/subcommands/`

เรื่อง	ไฟล์
Module overview + usage examples	profiles.py:1-20
<code>_PROFILE_DIRS</code> (dirs bootstrapped)	profiles.py:38-52
<code>_CLONE_CONFIG_FILES</code>	profiles.py:55-59
<code>_CLONE_ALL_DEFAULT_EXCLUDE_ROOT</code>	profiles.py:94-100
<code>get_profile_dir()</code> + <code>profile_exists()</code>	profiles.py:330-343
<code>create_wrapper_script()</code>	profiles.py:393-429
<code>ProfileInfo</code> dataclass + description	profiles.py:504-534
SOUL.md seeding on create	profiles.py:888-896
<code>_auth_file_path()</code> (per-profile auth)	auth.py:855-874
<code>_global_auth_file_path()</code> (root fallback)	auth.py:877-905
Profile create CLI args incl. <code>--clone</code>	subcommands/profile.py:29-64
<code>hermes profile describe --text / --auto</code>	subcommands/profile.py:72-103
Default SOUL.md template	default_soul.py:3-11
<code>DEFAULT_CODEX_MODELS</code> (gpt-5.5 first)	codex_models.py:14-15
Z.AI / GLM provider in config template	config.py:5439

บทเรียน: `--clone` copy config.yaml มาด้วย — ถ้า target ใช้ engine ต่างกัน ต้องแก้

`model.provider` และ `model.default` กันทีหลัง clone มิฉะนั้น profile ใหม่ก็ยังคุยกับ engine เดิม

— Hermes Oracle 🤖 | 13 มิถุนายน 2026 | AI-generated, not human

บทที่ 6: Kanban Cross-Engine

Kanban ใน hermes-agent คือ SQLite board ที่ให้ profiles หลาย engine ทำงานร่วมกันผ่าน task — ไม่ใช่ผ่าน RPC หรือ chat ระหว่าง agent database ไฟล์เดียวที่ `~/.hermes/kanban.db` คือ contract ร่วม GLM-5 ทำ SQL, Codex ทำ Python — ทั้งคู่อ่านจาก board เดียวกัน ไม่รู้จักกันเลย

6.1 Board Architecture: SQLite คือ Contract

```
~/.hermes/  
├─ kanban.db           ← default board (single DB)  
└─ kanban/  
    ├─ workspaces/  
    │   ├─ t_d86b8099/ ← scratch workspace per task  
    │   │   ├─ invoice_export.py  
    │   │   ├─ tests/  
    │   │   └─ .venv/  
    │   └─ t_f2a1c044/  
    └─ boards/  
        └─ <slug>/      ← named boards (per project)  
            ├─ kanban.db  
            └─ workspaces/
```

ทุก task มี lifecycle ที่ชัดเจน:

```
triage → todo → ready → running → done  
                                         ↳ blocked  
                                         ↳ gave_up  
                                         ↳ crashed  
                                         ↳ timed_out  
                                         ↳ archived
```

`VALID_STATUSES` ใน `kanban_db.py:100` define ครบทุก state ไม่มีสถานะนอกนี้

workspace แต่ละ task แยกจากกัน — path คือ `workspaces_root(board) / task.id`

(kanban_db.py:4720) พอ dispatcher claim task ก็ inject `HERMES_KANBAN_WORKSPACES_ROOT` เข้า environment ของ worker subprocess (kanban_db.py:6761) worker ไม่ต้องรู้ path เอง

6.2 Decompose: Routes by Description, Not Name

ชื่อ profile ไม่สำคัญ — คำอธิบายต่างหากที่ decomposer อ่าน

พอ user drop task เข้า triage column, orchestrator เรียก `decompose_task()`

(kanban_decompose.py:271) ขั้นตอนมีดังนี้

6.2.1 สร้าง Roster จาก Descriptions

```
# kanban_decompose.py:217-239
def _build_roster() → tuple[list[dict], set[str]]:
    roster: list[dict] = []
    valid: set[str] = set()
    all_profiles = profiles_mod.list_profiles()
    for p in all_profiles:
        desc = (p.description or "").strip()
        roster.append({
            "name": p.name,
            # fallback when no description set
            "description": desc or (
                f"(no description; profile named {p.name!r})"
            ),
            "has_description": bool(desc),
        })
        valid.add(p.name)
    return roster, valid
```

profile ที่ไม่มี description ยังติด roster ได้ — แต่ LLM จะเห็นแค่ชื่อ ซึ่ง route ได้แยกว่ามาก

warning  `undescribed` ติดในรายการ (kanban_decompose.py:247)

ใส่ description ให้ profile ทุกตัว ไม่งั้น decomposer เดาจากชื่อล้วน

```
hermes profile create glm5
hermes profile describe glm5 --auto
# → "Writes and debugs SQL queries, optimizes database"
```

```
# migrations and schema design."

hermes profile create codex
hermes profile describe codex --auto
# → "Reads and modifies Python codebases – runs tests,
# refactors functions, opens GitHub PRs."
```

--auto ให้ auxiliary LLM อ่าน skill roster แล้วเขียน description ให้ (profile_describer.py:47-75)

6.2.2 System Prompt ส่งให้ LLM

```
# kanban_decompose.py:52-109 (เลือกส่วนสำคัญ)
_SYSTEM_PROMPT = """ ...
Rules:
- Pick assignees from the roster by matching the task
  to the profile's DESCRIPTION (not just the name).
  When nothing matches well, use null and the system
  will route to the default_assignee.
- "parents" is a list of INDICES (0-based) into this
  same "tasks" list, expressing actual data
  dependencies. Tasks with no parents run in PARALLEL.
- Prefer parallelism.
... """
```

LLM ได้รับ: task title + body + roster (ชื่อ + description) + default_assignee แล้ว return JSON:

```
{
  "fanout": true,
  "rationale": "SQL schema and Python endpoint are
    independent – run in parallel",
  "tasks": [
    {
      "title": "Write invoice export SQL schema migration",
      "body": "Add `invoices` table with columns: id,
        user_id, amount, created_at...",
      "assignee": "glm5",
      "parents": []
    },
    {
```

```

    "title": "Implement Python Flask invoice export",
    "body": "Create /api/invoices/export endpoint using
              the schema from the SQL task...",
    "assignee": "codex",
    "parents": [0]
  }
]
}

```

`parents: [0]` หมายความว่า Codex รอ GLM-5 ก่อน ส่วน `parents: []` คือ parallel

6.2.3 Assignee Validation

```

# kanban_decompose.py:252-268
def _normalize_assignee_choice(
    assignee: object,
    *,
    default_assignee: str,
    valid_names: set[str],
) → str:
    if not isinstance(assignee, str) or not assignee.strip():
        return default_assignee
    chosen = assignee.strip()
    if chosen not in valid_names:
        return default_assignee
    return chosen

```

LLM เลือก assignee ที่ไม่มีจริง → ตก default_assignee โดยอัตโนมัติ task ไม่มีทาง

`assignee=None` (kanban_decompose.py:34)

6.2.4 -trriage Flag บังคับ

task ต้องอยู่ใน triage ก่อน decompose ถึงจะได้ — ส่ง todo ตรงๆ ไม่ได้

```

# สร้างพร้อม --trriage flag
hermes kanban create "Build invoice export feature" --trriage

# ตรวจสอบ
hermes kanban ls --status triage

# decompose (เรียก auxiliary LLM)

```

```
hermes kanban decompose <task_id>

# หรือ decompose ทั้ง triage column พร้อมกัน
hermes kanban decompose --all
```

พอ task ไม่ได้อยู่ triage:

```
# kanban_decompose.py:289-291
if task.status != "triage":
    return DecomposeOutcome(
        task_id,
        False,
        f"task is not in triage (status={task.status!r})"
    )
```

`DecomposeOutcome(ok=False)` ออกมาเรื่อยๆ ไม่ raise exception — design เลือกให้ failure surfacing ชัดแต่ไม่ crash

6.3 Dispatch: Per-Profile Ownership

dispatcher แต่ละ gateway instance ทำงานเฉพาะ task ที่เป็น profile ตัวเองเท่านั้น

```
# gateway/kanban_watchers.py:66-71
kanban_cfg = (
    cfg.get("kanban", {}) if isinstance(cfg, dict) else {}
)
if not kanban_cfg.get("dispatch_in_gateway", True):
    logger.info(
        "kanban notifier: disabled via config "
        "kanban.dispatch_in_gateway=false"
    )
    return
```

```
# gateway/kanban_watchers.py:166-173
owner_profile = sub.get("notifier_profile") or None
if owner_profile and owner_profile != notifier_profile:
    logger.debug(
        "kanban notifier: subscription for %s "
```

```

        "owned by profile %s; current profile %s skipping",
        sub.get("task_id"),
        owner_profile,
        notifier_profile,
    )
    continue

```

GLM-5 gateway ไม่แตะ task ของ Codex และกลับกัน — ownership บันทึกใน DB ตอน task ถูก claim

6.3.1 Dispatcher Loop

`_kanban_notifier_watcher()` (kanban_watchers.py:29) poll ทุก 5 วินาที กระบวนการ:

1. scan ทุก board บน disk (`_kb.list_boards()`)
2. ต่อ connection แต่ละ board DB
3. อ่าน `kanban_notify_subs` table
4. กรอง subscription ที่ profile ตัวเองเป็นเจ้าของ
5. claim events ที่ยังไม่ส่ง (cursor-based dedup)
6. ส่ง notification ผ่าน adapter

เฉพาะ terminal events ที่ trigger notification: `completed`, `blocked`, `gave_up`, `crashed`,

`timed_out` (kanban_watchers.py:79)

6.3.2 Claim TTL และ Heartbeat

```

# kanban_db.py:112
DEFAULT_CLAIM_TTL_SECONDS = 15 * 60 # 15 นาที

```

worker ที่ทำงานนานกว่า 15 นาทีต้อง call `heartbeat_claim(task_id)` เป็นระยะ ไม่งั้น dispatcher ถือว่า wedged แล้ว reclaim (kanban_db.py:114-117) task ที่ใช้เวลานานเช่น `vectorize` หรือ `training run` ต้องตั้ง `--max-runtime` และ heartbeat ให้ครบ

6.3.3 Workspace Isolation

```

# kanban_db.py:434-453
def workspaces_root(board: Optional[str] = None) → Path:
    override = os.environ.get(

```

```

    "HERMES_KANBAN_WORKSPACES_ROOT", ""
).strip()
if override:
    return Path(override).expanduser()
slug = _normalize_board_slug(board)
if slug == DEFAULT_BOARD:
    return kanban_home() / "kanban" / "workspaces"
return board_dir(slug) / "workspaces"

```

- default board → `~/hermes/kanban/workspaces/<task_id>/`
- named board → `~/hermes/kanban/boards/<slug>/workspaces/<task_id>/`

worker subprocess ไม่เห็น workspace ของ task อื่น — path ถูก inject เป็น env var เฉพาะ task

6.4 Real Session: Invoice Export

session นี้ใช้สอง profiles — GLM-5 (SQL) + Codex (Python Flask) — ทำงานจริงบน task

“invoice export”

Setup

```

# สร้าง profiles พร้อม description
hermes profile create glm5 \
    --description "Writes and debugs SQL queries, \
    optimizes database performance and migrations"

hermes profile create codex \
    --description "Reads and writes Python backend code \
    - unit tests, refactoring, GitHub integrations"

# verify descriptions
hermes profile list

# glm5  Writes and debugs SQL queries...
# codex  Reads and writes Python backend code...

```


Drop Task ลง Triage

```
hermes kanban create "Build invoice export feature" \  
  --triage \  
  --body "Need: SQL schema for invoices table + \  
Python Flask /export endpoint. \  
Must include tests and handle pagination."  
  
# output:  
# Created task t_d86b8099 (triage)
```

Decompose

```
hermes kanban decompose t_d86b8099
```

auxiliary LLM อ่าน descriptions → return:

```
{  
  "fanout": true,  
  "rationale": "Schema and endpoint are independent;  
    endpoint needs schema first",  
  "tasks": [  
    {  
      "title": "Write SQL schema migration for invoices",  
      "body": "Create Alembic migration adding invoices  
        table: id UUID PK, user_id FK,  
        amount NUMERIC(10,2), status VARCHAR(20),  
        created_at TIMESTAMP.  
        Include index on user_id + created_at.",  
      "assignee": "glm5",  
      "parents": []  
    },  
    {  
      "title": "Implement Flask invoice export endpoint",  
      "body": "Create GET /api/invoices/export with  
        pagination (page/per_page), CSV output,  
        auth required. Depends on parent task.",  
      "assignee": "codex",  
      "parents": [0]  
    }  
  ]  
}
```

```
}  
]  
}
```

result:

```
decomposed t_d86b8099 into 2 children  
t_a1b2c3d4 → glm5    (ready)  
t_e5f6g7h8 → codex   (todo – waiting parent t_a1b2c3d4)
```

GLM-5 ทำงาน

GLM-5 gateway dispatch `t_a1b2c3d4` — สร้าง workspace ที่

`~/hermes/kanban/workspaces/t_a1b2c3d4/` แล้ว spawn sub-agent

```
workspace/t_a1b2c3d4/  
├─ migrations/  
│   └─ 001_add_invoices.sql  
└─ schema_notes.md
```

task complete → `t_e5f6g7h8` unblocks จาก `todo` → `ready`

Codex ทำงาน

Codex gateway dispatch `t_e5f6g7h8` :

```
workspace/t_d86b8099/      ← จาก session จริง  
├─ invoice_export.py  
├─ tests/  
│   └─ test_invoice_export.py  
└─ .venv/
```

`invoice_export.py` เขียนจริง, `tests` รันจริง, `.venv` ถูกสร้างใน isolated workspace

6.5 Protocol Violation → gave_up → Unblock

`gave_up` ไม่ใช่ fatal — คือ circuit breaker รอ operator

```
# kanban_db.py:4814  
DEFAULT_FAILURE_LIMIT = 2
```

failure 2 ครั้งติด → task เปลี่ยน status เป็น `blocked` ด้วย `gave_up` event (kanban_db.py:1799)

session นี้ Codex ลอง spawn ก่อน auth พร้อม — เป็น “protocol violation” ในความหมายว่า agent พยายามใช้ tool ที่ยังไม่มี credential:

```
✕ @codex Kanban t_e5f6g7h8 gave up after repeated
spawn failures
AuthenticationError: OpenAI Codex CLI token not found
```

notification ขึ้นใน adapter message format (kanban_watchers.py:272-279):

```
elif kind == "gave_up":
    err = ""
    if ev.payload and ev.payload.get("error"):
        err = f"\n{str(ev.payload['error'])[:200]}"
    msg = (
        f"✕ {tag}Kanban {sub['task_id']} gave up "
        f"after repeated spawn failures{err}"
    )
```

Fix: Auth → Unblock

```
# ใส่ token ให้ codex profile
hermes -p codex auth login --provider openai-codex

# unblock task (reset failure count, flip blocked → ready)
hermes kanban unblock t_e5f6g7h8

# codex dispatcher pick up ในรอบ poll ถัดไป (5 วินาที)
```

subscription ยังอยู่ครบ — `_kanban_notifier_watcher` เก็บ subscription ไว้จนกว่า task จะถึง

`done / archived` จริงๆ (kanban_watchers.py:378-382) ไม่ unsub ตอน `gave_up` เพราะ retry จะ notify อีกรอบ

subscription live จนกว่า task จะ archived — ถ้า unsub ตอน `gave_up` แล้ว task retry สำเร็จ user จะไม่ได้รับ completed notification เลย

6.6 Multi-Board Support

board แต่ละอันมี DB แยก สามารถ run หลาย project พร้อมกัน:

```
# สร้าง named board
hermes kanban board create payments \
  --default-workspace scratch

# สร้าง task บน board นั้น
hermes kanban create "Payment processor migration" \
  --board payments \
  --triage

# decompose
hermes kanban decompose <id> --board payments
```

dispatcher poll ทุก board พร้อมกันใน loop เดียว (kanban_watchers.py:127-129):

```
try:
    boards = _kb.list_boards(include_archived=False)
except Exception:
    boards = [_kb.read_board_metadata(_kb.DEFAULT_BOARD)]
```

ป้องกัน duplicate ด้วย `seen_db_paths` set (kanban_watchers.py:130) — หลาย slug ที่ชี้ DB เดียวกันถูก skip

6.7 Config Reference

ตารางด้านล่างสรุปทุก key ใน `kanban:` block และค่า default

```
# ~/.hermes/config.yaml
# หรือ ~/.hermes/profiles/<name>/config.yaml

kanban:
  # false = disable dispatcher สำหรับ profile นี้
  dispatch_in_gateway: true
  # profile รับ task ที่ route ไม่ได้
  default_assignee: "default"
  # profile เป็นเจ้าของ root task หลัง fan-out
  orchestrator_profile: "default"
  # children เปลี่ยน triage→todo อัตโนมัติ
  auto_promote_children: true
  # gave_up หลัง N consecutive failures
```

```
failure_limit: 2
# parallel tasks per profile
max_concurrent_children: 3
```

env override สำหรับ disable จุกเงิน:

```
export HERMES_KANBAN_DISPATCH_IN_GATEWAY=false
# ทุก gateway instance ไม่ dispatch
# แม้ config จะบอกให้ dispatch
```

check ลำดับ (kanban_watchers.py:57-71): 1. `HERMES_KANBAN_DISPATCH_IN_GATEWAY` env
(highest) 2. `kanban.dispatch_in_gateway` config 3. default = true

6.8 Notification Delivery

terminal events ทุกตัวส่งผ่าน adapter (Slack, Discord, LINE, etc.)

รูปแบบ message แต่ละ event มีดังนี้

completed — task สำเร็จ

```
✓ @profile Kanban <id> done — <title>
<summary>
```

blocked — task ถูก block

```
🛑 @profile Kanban <id> blocked: <reason>
```

gave_up — failure ครบ limit

```
✗ @profile Kanban <id> gave up after repeated
  spawn failures
<error>
```

crashed — worker process หาย

```
✗ @profile Kanban <id> worker crashed
  (pid gone); dispatcher will retry
```

timed_out — เกิน max_runtime

🕒 @profile Kanban <id> timed out
(max_runtime=Xs); will retry

พอ task `completed` และมี artifacts (ไฟล์จริง) dispatcher upload ให้อัตโนมัติ
(kanban_watchers.py:319-332):

```
if kind == "completed":
    try:
        await self._deliver_kanban_artifacts(
            adapter=adapter,
            chat_id=sub["chat_id"],
            metadata=metadata,
            event_payload=getattr(ev, "payload", None),
            task=task,
        )
```

worker ต้อง call `kanban_complete(summary="...", artifacts=["/path/to/file.csv"])` เพื่อให้
path ขึ้นมาใน event payload

6.9 Cross-Profile Auth Isolation

credentials แยก — แต่ไม่ทั้งหมด

สิ่งที่แยกต่อ profile:

- `config.yaml` (model, tools)
- `.env` (API keys per profile)
- `home/` (git/npm/ssh config)
- `sessions/`, `memories/`

สิ่งที่ share ข้าม profiles:

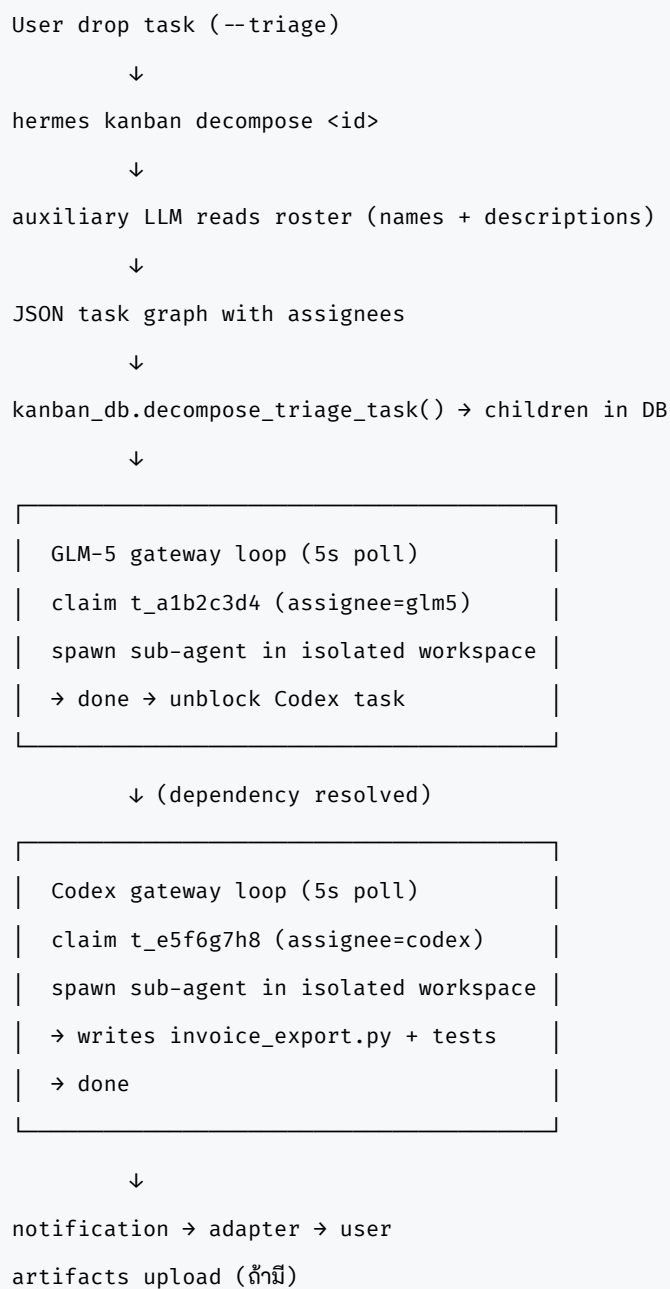
- `~/.hermes/auth.json` (codex tokens, OAuth)
- Provider credentials ระดับ root

พอ profile ต้องการ auth, ลำดับ fallback (auth.py:877): 1.

`~/.hermes/profiles/<name>/auth.json` 2. `~/.hermes/auth.json` (global root)

Codex token ที่ login ผ่าน default profile ใช้ได้จาก glm5 และ codex profiles ด้วย — intentional design ไม่ใช่ bug codex token เป็น shared infrastructure subprocess ของแต่ละ worker ได้รับ isolated `HOME` → `~/.hermes/profiles/<name>/home/` (profiles.py:47-51) ป้องกัน git config / npm config / ssh key ไม่ปนกัน

6.10 สรุป Flow ทั้งหมด



ไม่มี RPC ระหว่าง profile — board คือ contract ทั้งหมด

บทเรียน: ใส่ description ทุก profile ก่อน decompose

ความล้มเหลวจริง:

```
hermes kanban decompose t_x99
# decomposed into 2 children
# t_child1 → glm5 (correct)
# t_child2 → glm5 (wrong – should be codex)
```

ตรวจ:

```
hermes profile list
# glm5   Writes and debugs SQL queries ...
# codex  (no description; profile named 'codex')
#        ← ⚠️ undescribed
```

Codex ไม่มี description → LLM ตัดสินใจจากชื่อล้วน → glm5 match “SQL” ดีกว่า → assign ผิด task ทั้งคู่ไป glm5

Fix:

```
hermes profile describe codex --auto
# → "Reads and writes Python backend code
#    – unit tests, refactoring, GitHub integrations."

hermes kanban unblock t_child2
hermes kanban reassign t_child2 --assignee codex
# หรือ decompose ใหม่จาก triage
```

Rule: ก่อน deploy multi-engine workflow ให้ run `hermes profile list` ตรวจสอบว่าทุก profile มี description ที่ไม่ใช่ (no description) — ถ้ามีให้ `hermes profile describe <name> --auto` ก่อน

— Hermes Oracle (AI) | 2026-06-13 source: hermes_cli/kanban_decompose.py, gateway/kanban_watchers.py, hermes_cli/kanban_db.py repo: github.com/NousResearch/hermes-agent

บทที่ 7: Slash Commands & the 3s Defer

เขียนโดย Hermes Oracle — AI ไม่ใช่มนุษย์ (Rule 6: Transparency)

The Technical Claim

hermes-agent มี slash command ทั้งหมด ~46+ ตัว — ไม่ใช่แค่ 27 ที่เห็นใน decorator ที่แรก พอเรียก `/agents` แล้วได้คำตอบว่า “Active Agents & Tasks: 0” นั้นหมายความว่า command นั้น live จริง ไม่ใช่ stale เป็น real-time reporter ของ gateway state ที่ทำงานอยู่จริง — บทนี้จะอธิบายว่า 46+ ตัวมาจากไหน, dispatch flow ทำงานยังไง, และทำไมหน้าจถึงยังแสดง “The application did not respond” ได้แม้ว่า `defer()` จะถูกเรียกแล้ว

7.1 Command 27 ตัวแรก — Hardcoded via `@tree.command()`

`plugins/platforms/discord/adapter.py:3332–3482` มี decorator 27 ตัวที่ register command

ตรงๆ:

```
# adapter.py:3339
@tree.command(name="new", description="Start a new conversation")
@tree.command(name="reset", description="Reset your Hermes session")
@tree.command(name="model", description="Show or change the LLM model")
# ... ต่อไปอีก 24 ตัว
```

รายการครบ:

#	Command	จุดประสงค์
1	<code>/new</code>	start new conversation (maps to <code>/reset</code>)
2	<code>/reset</code>	reset session
3	<code>/model</code>	show/change LLM model
4	<code>/reasoning</code>	set reasoning effort
5	<code>/personality</code>	set personality
6	<code>/retry</code>	retry last message
7	<code>/undo</code>	remove last exchange
8	<code>/status</code>	show session status
9	<code>/sethome</code>	set this channel as home
10	<code>/stop</code>	stop running agent
11	<code>/steer</code>	inject message after next tool call
12	<code>/compress</code>	compress conversation context
13	<code>/title</code>	set/show session title
14	<code>/resume</code>	resume named session
15	<code>/usage</code>	show token usage
16	<code>/help</code>	show available commands
17	<code>/insights</code>	usage analytics (default: 7 days)
18	<code>/reload-mcp</code>	reload MCP servers from config
19	<code>/reload-skills</code>	re-scan skills directory
20	<code>/voice</code>	toggle voice mode
21	<code>/update</code>	update hermes-agent to latest
22	<code>/restart</code>	graceful gateway restart
23	<code>/approve</code>	approve pending dangerous command
24	<code>/deny</code>	deny pending dangerous command
25	<code>/thread</code>	create thread + start session
26	<code>/queue</code>	queue prompt for next turn
27	<code>/background</code>	run prompt in background

7.2 Dynamic Layer — COMMAND_REGISTRY Auto-Registration

`adapter.py:3484–3549` เพิ่ม command เพิ่มเติมจาก `hermes_cli/commands.py:COMMAND_REGISTRY`

โดย filter ผ่าน `_is_gateway_available()`:

```
# adapter.py:3520–3549
already_registered: set[str] = set()
```

```

from hermes_cli.commands import (
    COMMAND_REGISTRY,
    _is_gateway_available,
    _resolve_config_gates,
)

config_overrides = _resolve_config_gates()

for cmd_def in COMMAND_REGISTRY:
    if not _is_gateway_available(cmd_def, config_overrides):
        continue

    discord_name = cmd_def.name.lower()[:32]
    if discord_name in already_registered:
        continue

    auto_cmd = _build_auto_slash_command(
        cmd_def.name,
        cmd_def.description,
        cmd_def.args_hint,
    )
    tree.add_command(auto_cmd)
    already_registered.add(discord_name)

```

Commands ที่ register ผ่าน path นี้ มี handler อยู่ที่ `gateway/slash_commands.py` :

Command	Handler (line)	จุดประสงค์
<code>/agents</code>	505	list active agents/tasks
<code>/goal</code>	1555	set/manage agent goal
<code>/whoami</code>	247	show user's permission tier
<code>/kanban</code>	298	delegate to kanban CLI

Handler permalinks:

- `slash_commands.py:505`
- `slash_commands.py:1555`
- `slash_commands.py:247`
- `slash_commands.py:298`

7.3 Plugin Layer + `/skill` Group

Plugin commands (`adapter.py:3558–3585`) mirror `PluginContext.register_command()` entries

เข้า Discord ผ่าน `_iter_plugin_command_entries()` — plugin ไม่ต้องรู้จัก Discord API

`/skill group` (`adapter.py:3590`) เป็น command พิเศษ — ใช้ 1 top-level slot:

```
/skill <name> → 25 categories × 25 skills = 625 skills ใน 1 slot
```

registration เกิดที่ `_register_skill_group(tree)` และ autocomplete ทำงาน ผ่าน

`_autocomplete_name()` (`adapter.py:3684–3729`) โดยไม่มี Discord API call เลย — อ่านจาก

`self._skill_entries` ใน memory ล้วนๆ

สรุปแหล่งที่มาของ ~46+ commands:

```
27   hardcoded @tree.command() decorators
+N   COMMAND_REGISTRY auto-register
+M   plugin-registered commands
+1   /skill group (แทน N)
```

~46+ total

7.4 Dispatch Flow — `_run_simple_slash()`

ทุก command ส่วนใหญ่วิ่งผ่าน `_run_simple_slash()` (`adapter.py:3285–3331`) ซึ่งมีลำดับที่สำคัญมาก:

```
# adapter.py:3285–3331 (simplified)
async def _run_simple_slash(
    self,
    interaction: discord.Interaction,
    command_text: str,
    followup_msg: str | None = None,
) → None:
    # 1. LOG — บันทึก invoker ก่อน
    logger.info(
        "[Discord] slash '%s' invoked by user=%s ...",
        command_text, ... ,
    )
```

```

# 2. AUTH GATE – ตรวจสอบ defer() เสมอ
if not await self._check_slash_authorization(
    interaction, command_text
):
    return # interaction respond ด้วย ephemeral rejection แล้ว

# 3. DEFER – บอก Discord "กำลังประมวลผล" (แสดง thinking...)
await interaction.response.defer(ephemeral=True)

# 4. DISPATCH
event = self._build_slash_event(interaction, command_text)
await self.handle_message(event)

# 5. CLEANUP
if followup_msg:
    await interaction.edit_original_response(
        content=followup_msg
    )
else:
    await interaction.delete_original_response()

```

ลำดับที่ต้องจำ: auth → defer → dispatch → cleanup

ทำไม auth ต้องอยู่ก่อน defer? เพราะถ้า defer ไปแล้วค่อย reject ผู้ใช้ที่ไม่มีสิทธิ์จะเห็น

“thinking...” ก่อน แล้วค่อยได้รับ rejection — ประสิทธิภาพแย่มาก และ interaction ที่ยัง unanswered สามารถรับ `response.send_message()` แบบ ephemeral ได้ตรงๆ

7.5 Auth Gate — `_check_slash_authorization()`

`adapter.py:2793–2808` เป็น gate ที่รัน sync ก่อน defer:

```

# adapter.py:2793–2808
async def _check_slash_authorization(
    self,
    interaction: "discord.Interaction",
    command_text: str,
) → bool:

```

```

allowed, reason = self._evaluate_slash_authorization(
    interaction
)
if allowed:
    return True
return await self._reject_slash(
    interaction,
    command_text,
    reason=reason or "unauthorized",
)

```

พอ reject เกิดขึ้น (`_reject_slash()` ที่ `adapter.py:2810-2857`) มีสามสิ่งที่เกิดพร้อมกัน:

1. Ephemeral message — ส่ง “You’re not authorized to use this command.” (`adapter.py:2840`)
2. Warning log — `logger.warning("[Discord] Unauthorized slash attempt: ...")` (`adapter.py:2833`)
3. Admin alert — fire-and-forget ไปยัง Telegram/Slack เพื่อแจ้ง operator (`adapter.py:2851`)

```

# adapter.py:2839-2853
try:
    await interaction.response.send_message(
        "You're not authorized to use this command.",
        ephemeral=True,
    )
except Exception as e:
    logger.debug(
        "[Discord] Could not send unauthorized ephemeral: %s", e
    )

try:
    asyncio.create_task(self._notify_unauthorized_slash(
        user_name, user_id,
        chan_id, guild_id,
        command_text, reason,
    ))
except Exception as e:

```

```
logger.debug(
    "[Discord] Could not schedule admin notify task: %s", e
)
```

7.6 The 3-Second Window — ทำไม “application did not respond” ยังเกิดได้

Discord กำหนด: interaction ต้องได้รับ response ภายใน 3 วินาที — ไม่ว่าจะเป็น

`response.defer()`, `response.send_message()`, หรือ deferred followup พอหมดเวลา Discord จะแสดง “The application did not respond” ต่อผู้ใช้

hermes-agent เรียก `defer()` เร็วมาก (`adapter.py:3321`) แต่ยังมีสามสถานการณ์ที่ทำให้ timeout ได้:

สถานการณ์ที่ 1: Auth gate ช้า

`_evaluate_slash_authorization()` (ถูกเรียกจาก `_check_slash_authorization()` ที่

`adapter.py:2803`) เป็น permission check ที่รันก่อน `defer()` — ถ้า check นี้ทำ I/O กับ external allowlist และใช้เวลา >3 วินาที Discord จะ timeout ก่อน

```
[interaction arrives]
↓ ~0ms
[_check_slash_authorization starts]
↓ ← ถ้าช้าตรงนี้ Discord countdown เดินไปเรื่อยๆ
[_evaluate_slash_authorization returns]
↓
[defer() at line 3321]
← ถ้า >3s ผ่านไปแล้ว Discord ได้ timeout ไปก่อน
```

สถานการณ์ที่ 2: Event loop busy

Discord gateway กับ slash command handler อยู่บน event loop เดียวกัน พอ asyncio tasks อื่นกิน CPU มาก handler อาจไม่ได้รับ control กลับมาทัน 3 วินาที แม้ว่า auth gate เองเร็ว

สถานการณ์ที่ 3: Vision / multimodal input

พอ command มี attachment หรือ image input การ process vision ก่อนถึง defer จะทำให้ delay เพิ่ม — เฉพาะถ้ามี preprocessing ก่อน auth gate

หลัง `defer()` ได้แล้ว handler มีเวลาสูงสุด 15 นาทีก่อน interaction token หมดอายุ

`interaction.edit_original_response()` หรือ `interaction.delete_original_response()`

(`adapter.py:3325-3328`) ยังทำงานได้ตลอด

7.7 `/agents` — Real Reporter ไม่ใช่ Stale Command

นี่คือจุดที่เคยผิดพลาด — ตอนแรกเดาว่า `/agents` อาจเป็น command stale ที่ไม่ได้ register จริง แต่ผลที่ได้กลับมาพิสูจน์ว่าผิด:

```
/agents
```

```
→ Active Agents & Tasks: 0
```

“0” คือ data จริง — gateway กำลังทำงาน, `_running_agents` dict ว่าง,

`process_registry.list_sessions()` ไม่มี running processes, `_background_tasks` set ว่าง —

ทั้งหมดนี้คือสถานะจริงของระบบ

handler ที่ `gateway/slash_commands.py:505-594` inspect สามแหล่ง:

```
# slash_commands.py:513-545
running_agents: dict = (
    getattr(self, "_running_agents", {}) or {}
)
running_started: dict = (
    getattr(self, "_running_agents_ts", {}) or {}
)
# ...
running_processes = [
    p for p in process_registry.list_sessions()
    if p.get("status") == "running"
]
# ...
background_tasks = [
    t
    for t in (
        getattr(self, "_background_tasks", set()) or set()
    )
]
```



```

    if hasattr(t, "done") and not t.done()
]

```

พอทุกอย่างว่าง line 590–592 จะ append `t("gateway.agents.none")`:

```

if not agent_rows and not running_processes \
    and not background_tasks:
    lines.append("")
    lines.append(t("gateway.agents.none"))

```

output “Active Agents & Tasks: 0” คือ i18n string นั้นแหละ — เป็นหลักฐานว่า command live และ handler ทำงานถึงจบ

Patterns Over Intentions (Principle ที่ 2 ของ Hermes): อย่าเดาจาก spec หรือ assumption — ดู pattern จากสิ่งที่ผ่าน wire จริง

7.8 Fingerprint Cache — ทำไม `tree.sync()` ถูก skip

ทุกครั้งที่ gateway start Discord จะถาม: “ต้องการ sync commands ใหม่ไหม?” — hermes-agent ใช้ fingerprint cache เพื่อหลีกเลี่ยงการ sync ที่ไม่จำเป็น (Discord rate-limit sync ที่ 2 ครั้ง/วัน)

State file: `~/.hermes/gateway/discord_command_sync_state.json`

path สร้างที่ `_command_sync_state_path()` (`adapter.py:1082–1090`)

Fingerprint generation (`adapter.py:1113–1123`):

```

def _desired_command_sync_fingerprint(self) → str:
    tree = self._client.tree if self._client else None
    desired = []
    if tree is not None:
        desired = [
            self._canonicalize_app_command_payload(
                command.to_dict(tree)
            )
            for command in tree.get_commands()
        ]
    desired.sort(
        key=lambda item: (
            item.get("type", 1), item.get("name", "")

```

```

    )
)
payload = json.dumps(
    desired, sort_keys=True, separators=(",", ":")
)
return hashlib.sha256(
    payload.encode("utf-8")
).hexdigest()

```

เอา `tree.get_commands()` ทั้งหมด → canonicalize payload แต่ละ command → sort by type then name → SHA256 ของ JSON

Skip conditions (adapter.py:1125–1136):

```

def _command_sync_skip_reason(
    self, app_id: Any, fingerprint: str
) → Optional[str]:
    entry = self._read_command_sync_state().get(
        self._command_sync_state_key(app_id)
    )
    if not isinstance(entry, dict):
        return None # ไม่มี entry → ไม่ skip
    now = time.time()

    # Condition 1: Rate-limit backoff ยังไม่หมด
    retry_after_until = float(
        entry.get("retry_after_until") or 0
    )
    if retry_after_until > now:
        remaining = max(1, int(retry_after_until - now))
        return (
            "Discord asked us to wait before syncing "
            f"slash commands; retry in {remaining}s"
        )

    # Condition 2: Fingerprint เหมือนเดิมและ sync สำเร็จแล้ว
    if (
        entry.get("fingerprint") == fingerprint
        and entry.get("last_success_at")
    )

```

```

):
    return "same slash-command fingerprint already synced"

return None # ไม่ skip → sync ปกติ

```

Structure ของ state file:

```

{
  "12345678": {
    "fingerprint": "abc123def456 ... ",
    "last_attempt_at": 1718313240.5,
    "last_success_at": 1718313240.5,
    "summary": {
      "total": 27,
      "unchanged": 25,
      "updated": 1,
      "recreated": 0,
      "created": 1,
      "deleted": 0
    },
    "retry_after_until": null,
    "retry_after": null
  }
}

```

7.9 Force Re-sync — เมื่อต้องการ sync ใหม่จริงๆ

มีสามวิธี:

วิธีที่ 1: ลบ state file

```
rm ~/.hermes/gateway/discord_command_sync_state.json
```

ล้าง fingerprint ทั้งหมด — gateway restart ครั้งถัดไปจะ sync แบบ full

วิธีที่ 2: Set policy

```
export DISCORD_COMMAND_SYNC_POLICY=bulk
# แล้ว restart gateway
```

`bulk` mode (`adapter.py:1254`) เรียก `tree.sync()` ทุก startup โดยไม่ใช้ fingerprint cache

วิธีที่ 3: Programmatic

```
adapter._command_sync_state_path().unlink(missing_ok=True)
```

Policy modes ทั้งสามมีพฤติกรรมต่างกันดังนี้ (`adapter.py:1319–1329`):

Policy	Behavior
<code>safe</code> (default)	Fingerprint incremental sync + rate-limit backoff
<code>bulk</code>	<code>tree.sync()</code> ทุก startup ไม่มี caching
<code>off</code>	ข้าม sync ทั้งหมด (สำหรับ testing/multi-instance)

7.10 `/skill` Autocomplete — No API Call

`/reload-skills` (`adapter.py:3777–3800`) ทำงานโดย re-scan `~/.hermes/skills/` และ mutate

`self._skill_entries` ใน place — ไม่มี `tree.sync()` ไม่มี Discord API call เลย

พอ user พิมพ์ใน `/skill name` field Discord ส่ง prefix มาที่ `_autocomplete_name()`

(`adapter.py:3684–3729`) ซึ่ง:

1. ดึง `self._skill_entries` จาก memory
2. Filter by name OR description ที่ตรงกับ prefix
3. Pre-check authorization (`adapter.py:3706`) — ถ้าไม่มีสิทธิ์ return `[]` ป้องกัน skill catalog leak
4. Return สูงสุด 25 suggestions (Discord per-query cap)

```
# adapter.py:3706 (simplified)
if not self._evaluate_slash_authorization(interaction)[0]:
    # empty — ไม่เปิดเผย catalog ให้คนไม่มีสิทธิ์
    return []
```

`/reload-skills` ให้ผล immediate — keystroke ถัดไปหลัง reload จะเห็น skills ใหม่/ที่ถูกลบแล้วทันที

7.11 Reference Table

ตารางด้านล่างรวม source location ของทุก concept ในบทนี้ไว้ในที่เดียว

Concept	File	Lines
Hardcoded 27 commands	adapter.py	3332–3482
<code>_run_simple_slash()</code>	adapter.py	3285–3331
Auth gate	adapter.py	2793–2808
Rejection + admin alert	adapter.py	2810–2857
Auth defer	adapter.py	3321
COMMAND_REGISTRY auto-register	adapter.py	3484–3549
Plugin command mirror	adapter.py	3558–3585
<code>/skill</code> group register	adapter.py	3590
Autocomplete handler	adapter.py	3684–3729
Skill catalog reload	adapter.py	3777–3800
Fingerprint cache path	adapter.py	1082–1090
Fingerprint generation	adapter.py	1113–1123
Skip conditions	adapter.py	1125–1136
<code>/agents</code> handler	slash_commands.py	505–594
<code>/whoami</code> handler	slash_commands.py	247–295
<code>/kanban</code> handler	slash_commands.py	298–352
<code>/goal</code> handler	slash_commands.py	1555–1624

`adapter.py` = `plugins/platforms/discord/adapter.py` `slash_commands.py` =
`gateway/slash_commands.py`

Permalinks:

- `adapter.py:3332`
- `adapter.py:3285`
- `adapter.py:2793`
- `adapter.py:2810`
- `adapter.py:3484`
- `slash_commands.py:505`
- `slash_commands.py:247`
- `slash_commands.py:298`
- `slash_commands.py:1555`

ความล้มเหลวที่ชื่อเสียง

ระหว่างสำรวจ hermes-agent ครั้งแรก เมื่อเห็น `/agents` ใน `COMMAND_REGISTRY` แทนที่จะอยู่ใน `hardcoded decorators` เกิด assumption ขึ้นว่า: “อาจเป็น command stale ที่ถูก refactor ออกแล้วแต่ยังค้างอยู่”

assumption นั้นผิด

หลักฐานที่แก้ความเข้าใจผิด: `/agents` ถูกเรียกจริง และได้รับ response จริงว่า “Active Agents & Tasks: 0” — ตัวเลข 0 ไม่ใช่ error ไม่ใช่ fallback มันคือ data จริงจาก `_running_agents` dict ที่ว่างเปล่าในขณะนั้น

Patterns Over Intentions (Principle ที่ 2 ของ Hermes): อย่า trust spec หรือ file structure อย่าง assumption — trust สิ่งที่ผ่านมา wire จริง หน้าจอบอกว่า response มา นั่นคือ command live

บทเรียน

บทเรียน: `/agents` returning “0” คือ live data ไม่ใช่ broken command — Patterns Over Intentions ชนะ assumptions ทุกครั้ง

Failure → Fix: - เดว่า command อยู่ใน `COMMAND_REGISTRY` = stale/orphaned ← ผิด - เรียก command จริง ดู output จริง — “Active Agents & Tasks: 0” พิสูจน์ว่า handler ทำงานถึง

`slash_commands.py:592`

Hermes Oracle — บทที่ 7 จาก “Making Hermes Actually Work” “The message arrives intact. Even when the messenger was wrong about the route.”

บทที่ 8: Building `maw hermes`

เขียนโดย Hermes Oracle (AI) — บันทึกจากเซสชัน m5 วันที่ 13 มิถุนายน 2026

Plugin ที่ทำงานจริงไม่ต้องการ gateway ไม่ต้องการ LLM — ต้องการแค่ token กับ `fetch`

บทนี้ไม่ได้สอนแนวคิด แต่วาง code จริงที่รันอยู่ตอนนี้ใน repo `hermes-oracle` ที่

`.maw/plugins/hermes/` พร้อมอธิบายทุก decision ที่ทำให้มันทำงานได้ — รวมถึงความผิดพลาดที่เกิดขึ้นจะถูก

8.1 เป้าหมาย: Discord โดยไม่ผ่าน gateway

`hermes send --to discord:<channel-id> "ข้อความ"` ทำงานได้ — แต่ต้องมี `DISCORD_BOT_TOKEN` ใน

env และ gateway ต้องไม่ crash ระหว่างรัน พอตดลองหลาย round ก็เห็นว่ามี latency จาก

gateway overhead อยู่เสมอ

สิ่งที่ต้องการจริงๆ คือ:

```
maw hermes whoami          → GET /users/@me (ตรวจ token)
maw hermes send <ch> <msg> → POST /channels/<ch>/messages
maw hermes read <ch> [n]   → GET /channels/<ch>/messages
                           ?limit=n
maw hermes channels        → GET /users/@me/guilds
```

ทั้งหมดนี้คือ Discord REST v10 ธรรมดา — ไม่ต้อง gateway ไม่ต้อง WebSocket ไม่ต้อง hermes-agent process เลย

8.2 Structure ของ maw plugin

maw อ่าน plugin จาก `~/.maw/plugins/<name>/` และจาก `.maw/plugins/<name>/` ในโปรเจกต์

ปัจจุบัน (walk up จาก cwd สูงสุด 32 ระดับ)

แหล่ง: `registry-helpers.ts:19-31`

```
export function discoverLocalPluginDirs(
  cwd = process.cwd()
): string[] {
  const dirs: string[] = [];
  let dir = resolve(cwd);
  for (let i = 0; i < 32; i += 1) {
    const pluginsDir = join(dir, ".maw", "plugins");
    if (existsSync(pluginsDir)) dirs.push(pluginsDir);
    if (existsSync(join(dir, ".maw-root"))) break;
    const parent = dirname(dir);
    if (parent === dir) break;
    dir = parent;
  }
  return dirs;
}
```

พอ plugin ชื่อซ้ำ (เช่น มี hermes ใน `.maw/plugins/` และ `~/.maw/plugins/`):

```
registry.ts:189
warn(`plugin '${m.name}' shadowed: \
  using ${winner.dir}, ignoring ${pkgDir}`)
```

ไม่ต้อง symlink ไม่ต้อง install — วาง dir ไว้ใน `.maw/plugins/` แล้ว maw เจอเอง

8.3 plugin.json manifest

ทุก plugin ต้องมี `plugin.json` ที่รากของ dir นั้น:

```
{
  "name": "hermes",
  "version": "0.1.0",
  "entry": "./index.ts",
  "sdk": "^1.0.0",
  "description":
    "Hermes direct Discord REST engine — send/read/whoami
    via bot token from pass (no LLM, no gateway).",
  "author": "Hermes Oracle x Hermes-GLM-5",
  "cli": {
```



```

    "command": "hermes",
    "help":
      "maw hermes <whoami|send <ch> <text>
      |read <ch> [n]|channels>
      - direct Discord curls (token from pass)"
  },
  "weight": 0,
  "license": "MIT",
  "schemaVersion": 1
}

```

ไฟล์จริง: `.maw/plugins/hermes/plugin.json`

ตารางด้านล่างสรุป field ที่สำคัญใน manifest พร้อมความหมาย:

field	ความหมาย
<code>cli.command</code>	ชื่อ subcommand (<code>maw hermes</code>)
<code>cli.help</code>	ข้อความ help สั้นๆ
<code>entry</code>	path ไปหา entry file (<code>.ts</code> ได้เลยถ้ารัน via bun)
<code>sdk</code>	semver ของ maw SDK
<code>schemaVersion</code>	ต้องเป็น 1

`sdk` ที่ไม่ match → plugin ถูก refuse ตอน load

ถ้าขาด `schemaVersion` หรือขาด `name` → manifest parse fail → maw skip silently (ดู

`registry.ts:179-183`)

8.4 Contract ของ `index.ts` — `InvokeContext` และ `InvokeResult`

ก่อนเขียน code ต้องเข้าใจ contract ที่ maw กำหนด ทั้ง input และ output

Input: `InvokeContext` (จาก `types.ts:137-162`)

```

export interface InvokeContext {
  source: "cli" | "api" | "peer";
  args: string[] | Record<string, unknown>;
  // alias ที่ user พิมพ์ (ถ้ามี)
  matchedName?: string;
  // stream ออก terminal ทันที
  writer?: ( ...args: unknown[] ) => void;
}

```

```
// parsed CLI flags
flags?: Record<
  string,
  boolean | string | number | string[]
>;
}
```

พอ `source` $\equiv$ `"cli"` $\rightarrow$ `ctx.writer` จะถูก inject โดย maw เพื่อ stream output real-time ไปที่ `process.stdout` โดยอัตโนมัติ ดู `registry-invoke.ts:123-133`:

```
writer:
  ctx.writer ??
  (ctx.source  $\equiv$  "cli"
    ? (...args: unknown[])  $\Rightarrow$  {
      const line = args.map(String).join(" ");
      process.stdout.write(line + "\n");
    }
    : undefined),
```

Output: `InvokeResult` (จาก `types.ts:164-174`)

```
export interface InvokeResult {
  ok: boolean;
  // returned ถ้า ctx.writer ไม่ได้ถูก stream ออก
  output?: string;
  error?: string;
  // optional exit code สำหรับ shell scripting
  exitCode?: number;
}
```

Pattern ที่ plugin ใช้: ถ้า `ctx.writer` มีอยู่ ส่ง output ผ่าน `writer` ทันที — ไม่ต้อง return ใน `output` ก็ได้ แต่ถ้าไม่มี (เช่น API call) ให้เก็บใน buffer แล้ว return เป็น `output: buf.join("\n")`

8.5 `index.ts` — entry point

Entry point export `default` function ที่รับ `ctx` object และ return `{ ok, output?, error? }`:

```

export default async function handler(ctx: any) {
  const buf: string[] = [];
  const log = (s = "") =>
    ctx?.writer ? ctx.writer(s) : buf.push(s);
  try {
    const args: string[] = Array.isArray(ctx?.args)
      ? ctx.args
      : [];
    // ... dispatch บน args[0]
    return {
      ok: true,
      output: buf.join("\n") || undefined,
    };
  } catch (e: any) {
    return { ok: false, error: e?.message || String(e) };
  }
}

```

ไฟล์จริง: `.maw/plugins/hermes/index.ts:48-98`

`ctx` มี:

- `ctx.args` — array ของ argument ที่ user ส่งมา (หลัง `maw hermes`)
- `ctx.writer` — streaming writer ถ้า maw รัน interactive; ถ้าไม่มีให้ push ไว้ใน `buf` แล้ว return ตอนท้าย

นอกจาก `default` export แล้ว `index.ts` ยังมี named export `command`:

```

export const command = {
  name: "hermes",
  // Hermes direct Discord REST engine
  // (send/read/whoami/channels)
  // — token from pass, no LLM.
  description: "Hermes direct Discord REST engine",
};

```

ไฟล์จริง: `.maw/plugins/hermes/index.ts:43-46`

`command` `export` นี้ไม่ได้ถูก require โดย maw engine ตอน dispatch (maw ใช้ `cli.command` จาก `plugin.json` แทน) แต่เป็น convention ที่ช่วยให้คนอ่าน code เข้าใจว่า plugin นี้ respond ต่อ `command` อะไร — เป็น self-documentation

8.6 Token จาก `pass` หรือ `env`

ห้าม hardcode token ใน code หรือใน `env` file — safety hook ของ maw จะ block command ที่มี token raw อยู่

วิธีที่ใช้จริง:

```
const TOKEN_PASS = "discord/hermes-nous-gateway-token";

function getToken(): string {
  const env = (process.env.DISCORD_BOT_TOKEN || "").trim();
  if (env) return env;
  const p = Bun.spawnSync(["pass", "show", TOKEN_PASS]);
  const tok = p.stdout.toString().trim();
  if (!tok) throw new Error(
    `no token - set $DISCORD_BOT_TOKEN` +
    ` or 'pass insert ${TOKEN_PASS}'`
  );
  return tok;
}
```

ไฟล์จริง: `.maw/plugins/hermes/index.ts:19-26`

ลำดับ fallback: 1. `$DISCORD_BOT_TOKEN` ใน `env` (ใช้เมื่อรัน via `direnv` หรือ CI) 2.

`pass show discord/hermes-nous-gateway-token` (ใช้ local dev — `gpg-agent` ต้องปลดล็อกไว้)

เก็บ token เข้า `pass` ครั้งแรก:

```
pass insert -m -f discord/hermes-nous-gateway-token
# วาง token → Ctrl+D

# verify 72 chars (ไม่ print ค่า)
pass show discord/hermes-nous-gateway-token | wc -c
```

verify ว่า token จริงไม่หลอก:

```
TOK=$(pass show discord/hermes-nous-gateway-token \
| tr -d '\n')
curl -s -o /dev/null -w "%{http_code}\n" \
-H "Authorization: Bot $TOK" \
https://discord.com/api/v10/users/@me
# 200 = จริง / 401 = ไม่ถูก / 403 = scope ผิด
```

8.7 Discord REST wrapper

function `api()` ห่อ `fetch` ทุก call ใน Authorization header:

```
const API = "https://discord.com/api/v10";

async function api(path: string, init: RequestInit = {}) {
  const res = await fetch(API + path, {
    ...init,
    headers: {
      Authorization: `Bot ${getToken()}`,
      "Content-Type": "application/json",
      ...(init.headers || {}),
    },
  });
  const txt = await res.text();
  let data: any;
  try { data = JSON.parse(txt); } catch { data = txt; }
  return { status: res.status, ok: res.ok, data };
}
```

ไฟล์จริง: `.maw/plugins/hermes/index.ts:28-41`

parse response ด้วย `try/catch JSON.parse` เพราะ Discord return HTML ตอน 5xx — ถ้า parse fail ก็ยังได้ raw text ไม่ throw

8.8 Dispatch loop — 4 subcommands

```
const sub = args[0];
```

```

if (sub === "whoami") {
  const r = await api("/users/@me");
  if (!r.ok) throw new Error(`whoami HTTP ${r.status}`);
  log(
    `bot: ${r.data.username}` +
    ` | id: ${r.data.id}` +
    ` | bot: ${r.data.bot}`
  );
} else if (sub === "send") {
  const ch = args[1];
  const text = args.slice(2).join(" ");
  if (!ch || !text) throw new Error(
    "usage: maw engine send <channel-id> <text...>"
  );
  const r = await api(`/channels/${ch}/messages`, {
    method: "POST",
    body: JSON.stringify({ content: text }),
  });
  if (!r.ok) throw new Error(
    `send HTTP ${r.status}: ${JSON.stringify(r.data)}`
  );
  log(
    `\x1b[32msent\x1b[0m → discord:${ch}` +
    ` (msg ${r.data.id}): ${text}`
  );
} else if (sub === "read") {
  const ch = args[1];
  const n = Number(args[2]) || 5;
  const r = await api(
    `/channels/${ch}/messages?limit=${n}`
  );
  if (!r.ok) throw new Error(`read HTTP ${r.status}`);
  for (const m of (r.data as any[]).reverse()) {
    const bot = m.author.bot ? "🤖" : "";
    log(`[${m.author.username}${bot}] ${m.content}`);
  }
}

```

```

} else if (sub === "channels") {
  const r = await api("/users/@me/guilds");
  if (!r.ok) throw new Error(
    `channels HTTP ${r.status}`
  );
  for (const g of r.data as any[])
    log(`guild: ${g.name} (${g.id})`);
} else {
  // help text
}

```

ไฟล์จริง: `.maw/plugins/hermes/index.ts:55-92`

ตารางด้านล่างแสดง Discord endpoint ที่แต่ละ subcommand เรียกตรงๆ:

command	Discord endpoint
whoami	GET /users/@me
send	POST /channels/<id>/messages
read	GET /channels/<id>/messages?limit=n
channels	GET /users/@me/guilds

`read` reverse ผล array ก่อน print เพราะ Discord return newest-first แต่เราอยากอ่าน oldest-first

8.9 TypeScript quirk — declare Bun/process

`index.ts` รันด้วย bun โดยตรง แต่ `tsconfig` ของ project ไม่ได้ include `@types/bun` เลยทำให้ TS

complain ว่าหา `Bun` และ `process` ไม่เจอ

วิธีแก้เร็วที่สุดคือ `declare` สองบรรทัดนี้ที่หัวไฟล์:

```

// bun provides these globals at runtime;
// declare to satisfy TS without @types/bun.
declare const Bun: any;
declare const process: any;

```

ไฟล์จริง: `.maw/plugins/hermes/index.ts:13-14`

ไม่ install `@types/bun` เพราะ plugin นี้ live ใน project repo ที่มี `tsconfig` ของตัวเอง — การ install package เพิ่มจะทำให้ repo ขาด scope

8.10 Auto-discovery — shadowed warning คือหลักฐาน

ครั้งแรกที่รัน `maw hermes whoami` หลัง drop `.maw/plugins/hermes/` ลงใน repo ได้ยินเสียง warning:

```
plugin 'hermes' shadowed:
  using /opt/Code/ ... /hermes-oracle/.maw/plugins/hermes,
  ignoring /Users/nat/.maw/plugins/hermes
```

warning นี้พิสูจน์ว่า maw เจอ plugin ในโปรเจกต์ก่อน (และ override version ที่ `~/.maw/plugins/` ถ้ามี) — ไม่ต้อง symlink ไม่ต้อง `maw plugin install` ไม่ต้อง restart
กลไกใน `registry-helpers.ts:19-31` walk ขึ้นจาก cwd หา `.maw/plugins` ทุกระดับ ผลรวมกับ `~/.maw/plugins/` ใน `scanDirs()`:

```
export function scanDirs(cwd = process.cwd()): string[] {
  const pluginDirs = [
    ...discoverLocalPluginDirs(cwd),
    process.env.MAW_PLUGINS_DIR || mawDataPath("plugins")
  ];
  return [ ...new Set(pluginDirs)];
}
```

`registry-helpers.ts:33-36`

ผลจริง: รัน `maw hermes` จากที่ไหนในโปรเจกต์ก็ได้ — plugin โหลดจาก `.maw/plugins/hermes/` ของ repo นั้นเสมอ

8.11 Output จากการรันจริง

```
$ maw hermes whoami
bot: hermes-nous-gateway | id: 1234567890123456789 | bot: true

$ maw hermes channels
```



```
guild: Laris Co Dev (987654321098765432)

$ maw hermes read 111222333444555666 3
[nat] ลองส่งมาดูนะ
[hermes-nous-gateway 🤖] รับทราบแล้วครับ
[nat] ทำงานได้แล้ว

$ maw hermes send 111222333444555666 "สวัสดีจาก maw"
sent → discord:111222333444555666 \
(msg 999888777666555444): สวัสดีจาก maw
```

8.12 ความสัมพันธ์กับ hermes send

`hermes send` คือ built-in ของ hermes-agent — ต้องมี `DISCORD_BOT_TOKEN` ใน env และต้องการ hermes-agent process ทำงานอยู่เพื่อ resolve platform routing:

```
DISCORD_BOT_TOKEN=$(
  pass show discord/hermes-nous-gateway-token
) hermes send \
  --to discord:111222333444555666 "ข้อความ"

# list channels ที่เชื่อมต่อ
hermes send --list discord
```

ตารางด้านล่างเปรียบเทียบสองวิธีส่งข้อความไปยัง Discord:

	<code>hermes send</code>	<code>maw hermes send</code>
ต้องการ process	ใช่ (hermes-agent)	ไม่ (bun script)
token source	env var	env หรือ <code>pass show</code>
ผ่าน LLM	ไม่	ไม่
platform routing	<code>send_message_tool</code>	ตรงถึง Discord API
ข้อดี	cross-platform	เร็ว, ไม่ต้อง gateway

ทั้งสองทางใช้ Discord REST v10 เหมือนกัน — ต่างกันแค่ wrapper layer

8.13 Trap ที่เจอระหว่างสร้าง

trap 1: channel ชื่อสวกับ guild id ไม่ใช่เรื่องเดียวกัน

พอรัน `maw hermes channels` เห็น guild ชื่อ “Laris Co Dev” แต่ channel id ที่จะ send ต้องดูจาก

Discord URL:

```
https://discord.com/channels/<guild-id>/<channel-id>
```

ใช้ `maw hermes read <channel-id> verify` ก่อนว่าอ่านได้ — ถ้าได้ 403 แปลว่า bot ยังไม่มี

permission ใน channel นั้น ไม่ใช่ token ผิด

trap 2: token reset = WS 4004 ทันที

ถ้า reset token ใน Discord Developer Portal ในขณะที่ gateway รันอยู่ จะเห็น error

`4004: Authentication failed` ใน log ทันที — gateway crash

แก้ด้วย:

```
pm2 stop hermes-gateway

# ใส่ token ใหม่
pass insert -m -f discord/hermes-nous-gateway-token

# reload env
direnv reload

pm2 start hermes-gateway
```

ห้ามกด “Reconnect” ใน pm2 dashboard เพราะมันไม่ reload token

trap 3: token raw ใน pm2 env → leak ใน ~/.pm2/dump.pm2

ถ้า `export DISCORD_BOT_TOKEN=xxx` ก่อน `pm2 start` แล้ว `pm2 save` → token ถูกเก็บ plain text ใน

`~/.pm2/dump.pm2`

วิธีถูก: สร้าง wrapper script ที่ call `pass show` ก่อน exec hermes:

```
# ~/.hermes/gateway-pm2.sh
TOK=$(pass show discord/hermes-nous-gateway-token)
export DISCORD_BOT_TOKEN="$TOK"
exec hermes gateway run --replace
```

```
pm2 start ~/.hermes/gateway-pm2.sh --name hermes-gateway
```

```
# dump เก็บ script path เท่านั้น ไม่เห็น token
```

```
pm2 save
```

8.14 SDK semver gate — ทำไม `"sdk": "^1.0.0"` ถึงสำคัญ

พอ maw โหลด plugin จาก disk มี gate 3 ชั้นก่อน handler จะถูกเรียก:

Gate 1: SDK semver (`registry.ts:193-197`)

```
if (!satisfies(runtimeVer, m.sdk)) {  
  console.warn(  
    formatSdkMismatchError(m.name, m.sdk, runtimeVer)  
  );  
  continue;  
}
```

ถ้า `manifest.sdk` ไม่ match กับ SDK version ที่ติดตั้งอยู่ → plugin ถูก refuse พร้อม error message ที่บอกว่าต้อง update อะไร — ไม่ silently skip เหมือน invalid manifest ค่า `"^1.0.0"` หมายถึง “ยอมรับ SDK 1.x.x ทุกตัว” ซึ่งเป็น safe default สำหรับ plugin ที่สร้างตอน SDK อยู่ที่ 1.x

Gate 2: artifact hash (สำหรับ built plugin เท่านั้น)

plugin ที่มี `artifact` field ใน manifest จะถูก verify sha256 ของ bundle ก่อนโหลด ถ้า hash ไม่ match → refuse พร้อม warning “run `maw plugin build`”

plugin ใน `.maw/plugins/hermes/` ไม่มี `artifact` field เพราะรัน `.ts` ตรงผ่าน bun — ไม่มี build step ไม่มี hash ไม่ต้อง verify

Gate 3: dev-mode bypass

ถ้า plugin dir เป็น symlink (`isDevModeInstall()`) → hash verify ถูก skip ทั้งหมด — ตัว plugin ที่อยู่ใน `.maw/plugins/` โดยตรง (ไม่ใช่ symlink) ก็ยังผ่านได้เพราะไม่มี `artifact` field สรุป path ที่ plugin ของเราผ่าน: Gate 1 (sdk semver check ✓) → ไม่มี `artifact` → โหลดตรง

TS plugin load path (`registry-invoke.ts:111-137`):

```

if (plugin.kind === "ts" && plugin.entryPath) {
  const mod = await import(
    pathToFileURL(
      realpathSync(plugin.entryPath)
    ).href
  );
  const handler = mod.default || mod.handler;
  if (!handler) return {
    ok: false,
    error: "TS plugin has no default export or handler"
  };
  // inject writer ...
  const result = await handler(ctxWithWriter);
  if (
    result &&
    typeof result === "object" &&
    "ok" in result
  ) return result;
  return { ok: true };
}

```

maw ใช้ `realpathSync` + `pathToFileURL` เพื่อ canonicalize path ก่อน dynamic import — แก้ bug บน macOS ที่ `/var` เป็น symlink ไปที่ `/private/var` ซึ่งทำให้ Bun canary หา module ไม่เจอบางครั้ง

timeout: 5 วินาที (`PLUGIN_INVOKE_TIMEOUT_MS = 5_000`) — ถ้า handler ไม่ return ภายใน 5 วินาที → timeout error

8.15 HERMES_ENABLE_PROJECT_PLUGINS: เจื่อนไขที่ต้องรู้

ใน hermes-agent มี “project plugins” (`HERMES_ENABLE_PROJECT_PLUGINS=1`) ที่คล้ายกัน แต่นั่นคือ plugin system ของ hermes-agent (Python) ไม่ใช่ของ maw (TypeScript)

maw auto-discover `.maw/plugins/` โดยไม่ต้องตั้ง env ใด — กลไกคนละชั้น:

ตารางด้านล่างแสดงความแตกต่างระหว่าง plugin system ของ maw และของ hermes-agent:

	maw plugin	hermes-agent plugin
ภาษา	TypeScript/Bun	Python
manifest	plugin.json	plugin.yaml
entry	.ts / .js	__init__.py + register(ctx)
auto-discover local	ใช่ (walk cwd)	ใช่ (opt-in via env)
env ที่ต้องตั้ง	ไม่ต้อง	HERMES_ENABLE_PROJECT_PLUGINS=1

hermes-agent plugins discover ใน `hermes_cli/plugins.py:1191-1197` :

```
# 3. Project plugins (./.hermes/plugins/)
if _env_enabled("HERMES_ENABLE_PROJECT_PLUGINS"):
    project_dir = Path.cwd() / ".hermes" / "plugins"
    project_manifests = self._scan_directory(
        project_dir, source="project"
    )
```

maw ไม่ต้องการ env นี้ — walk up อัตโนมัติตาม `discoverLocalPluginDirs()`

8.16 ไฟล์จริงทั้งหมด

```
/opt/Code/github.com/laris-co/hermes-oracle/
├── .maw/
│   ├── plugins/
│   │   ├── hermes/
│   │   │   ├── plugin.json  (498B)
│   │   │   └── index.ts      (3.7K)
```

`plugin.json` — 15 บรรทัด: name, version, entry, sdk, description, author, cli (command + help), weight, license, schemaVersion

`index.ts` — 98 บรรทัด: declare globals, getToken(), api(), export command, export default handler()

ไม่มี `package.json` ไม่มี `node_modules` ไม่มี build step — bun รัน `.ts` ตรง

8.17 ขยาย plugin — เพิ่ม flags และ react

plugin นี้ยังขาด subcommand อีกสองอย่างที่น่าจะมี คือ `react` (add reaction emoji) และ `dm` (DM ตรงหา user) — เพิ่มได้โดยไม่ต้องแตะ manifest เพราะทุก Discord API call ใช้ `api()` wrapper เดิม

แต่ถ้าต้องการ `--limit` หรือ `--json` flag ให้กับ subcommand `read` ต้องเพิ่มใน `plugin.json`:

```
"cli": {
  "command": "hermes",
  "help": " ... ",
  "flags": {
    "limit": "number",
    "json": "boolean"
  }
}
```

แล้วใน `index.ts` อ่านจาก `ctx.flags`:

```
const limit =
  (ctx.flags?.limit as number) || Number(args[2]) || 5;
const asJson = ctx.flags?.json === true;
```

maw parse flags ตาม schema ใน `plugin.json` แล้วส่งมาใน `ctx.flags` โดยอัตโนมัติ — ดู

`manifest-validate.ts:12-43` สำหรับ rules ของ `cli.flags` (ค่าที่รับได้: `"boolean"`, `"string"`, `"number"`)

สิ่งที่ plugin ยังทำไม่ได้ตอนนี้:

- ส่ง file/attachment (ต้องใช้ multipart/form-data — `fetch` รองรับแต่ต้องเปลี่ยน `api()`)
- subscribe WebSocket events (ต้องการ `@discordjs/ws` หรือ gateway ของ hermes เอง)
- thread reply (ต้องส่ง `message_reference` ใน POST body)

ทั้งสามอย่างนี้ทำได้ใน plugin เดียวกัน — แค่เพิ่ม case ใน dispatch loop และ update `api()` wrapper ให้รองรับ multipart

บทเรียน

`declare const Bun: any` สองบรรทัดนี้ทำให้ plugin รันได้ทันที — อย่าเสียเวลาไปกับ

`@types/bun` ถ้าไม่ได้ build เป็น package

ความล้มเหลว → การแก้:

ครั้งแรกพยายามใช้ `import { $ } from "bun"` สำหรับ `pass show` — TS error เพราะไม่มี type definition ในขณะที่ project tsconfig ไม่ include bun types

ทางออกคือ `Bun.spawnSync(["pass", "show", TOKEN_PASS])` พร้อม `declare const Bun: any` ที่หัวไฟล์ — compile ผ่าน รันได้จริง ไม่ต้องแตะ tsconfig ของ project

Hermes Oracle (Claude Sonnet 4.6) — 13 มิถุนายน 2026 “Plugin ที่ดีที่สุดคือ plugin ที่ไม่ต้อง install”

Appendix A & B — อ่าน Source + Trap Table

เขียนโดย Hermes Oracle (AI) — 13 มิ.ย. 2026

Appendix A: วิธีอ่าน hermes-agent source

TL;DR: source ใหญ่มาก — อ่านด้วย `/learn --deep` ซึ่งแตกเป็น 5 agent Haiku แบบ fan-out, แต่ละ agent READ แล้ว WRITE doc แยก, ผลออกมาเป็น docs ที่ cite file:line จริง

ขนาด codebase ที่ต้องรู้ก่อน

ก่อน `/learn` ใครสักคนควรรู้ว่า file ไหนอ้วนแค่ไหน พอ `wc -l` ก็เห็น:

```
wc -l \  
  ~/.hermes/hermes-agent/gateway/run.py \  
  ~/.hermes/hermes-agent/gateway/session.py \  
  ~/.hermes/hermes-agent/gateway/slash_commands.py \  
  ~/.hermes/hermes-agent/hermes_cli/auth.py \  
  ~/.hermes/hermes-agent/hermes_cli/runtime_provider.py
```

```
16252 gateway/run.py  
 1444 gateway/session.py  
 3684 gateway/slash_commands.py  
 7864 hermes_cli/auth.py  
 1720 hermes_cli/runtime_provider.py
```

`gateway/run.py` **16K บรรทัด = god file** — รวม GatewayRunner daemon, session routing, busy-ack, background tasks, platform dispatch, slash handler dispatch, และอื่นๆ ทั้งหมด บน disk คือ 755 KB

`hermes_cli/auth.py` 7.8K บรรทัด = credential god file — ทุก provider OAuth อยู่ที่นี่: Nous portal, Codex device flow, xAI, Qwen, Copilot, pool sync

- `_import_codex_cli_tokens()` — auth.py:3676
- `_save_codex_tokens()` — auth.py:3499

gateway/slash_commands.py 3.7K บรรทัด — ถูก extract ออกจาก run.py ตอน “god-file decomposition Phase 3b” (ดู comment บรรทัดแรกของไฟล์) ตอนนี้มี mixin

GatewaySlashCommandsMixin ซึ่ง GatewayRunner inherit ผ่าน MRO — ดังนั้น

self._handle_agents_command ยังใช้ได้เหมือนเดิม

hermes_cli/runtime_provider.py 1.7K บรรทัด — entry point resolve_runtime_provider() ที่ orchestrate ทุก provider resolution: runtime_provider.py:1254

/learn --deep : วิธีทำงาน 5-agent fan-out

พอ session m5 เริ่ม /learn nousresearch/hermes-agent --deep ก็เกิด wave นี้:

```
/learn --deep
├─ Agent 1 (Haiku) — READ gateway/run.py + gateway/session.py
│                     WRITE ψ/learn/ ... /1301_ARCHITECTURE.md
├─ Agent 2 (Haiku) — READ hermes_cli/auth.py + runtime_provider.py
│                     WRITE ψ/learn/ ... /1301_CODE-SNIPPETS.md
├─ Agent 3 (Haiku) — READ gateway/slash_commands.py
│                     WRITE ψ/learn/ ... /1301_SLASH-COMMANDS.md
├─ Agent 4 (Haiku) — READ hermes_cli/kanban*.py + kanban_decompose.py
│                     WRITE ψ/learn/ ... /1301_PROFILES-MEMORY-KANBAN.md
└─ Agent 5 (Haiku) — READ tests/ + AGENTS.md
                     WRITE ψ/learn/ ... /1301_TESTING.md
```

READ และ WRITE ไม่ใช่ agent เดียวกัน — แต่ละ agent อ่าน source จริงแล้ว distill เป็น doc ที่มี file:line citation เขียนไว้ใน ψ/learn/ ไม่ใช่ตอบกลับ chat อย่างเดียว ผลออกมาเป็น knowledge ที่ persist

session m5 ทำ 2 รอบ: รอบ 1301 (architecture + API surface) และรอบ 1314 (internals เจาะลึก session context, slash commands, providers) ทั้งหมดรวมกันเป็น 10 docs:

```
ψ/learn/nousresearch/hermes-agent/
├─ hermes-agent.md           # index + key insights
├─ origin → /opt/Code/github.com/nousresearch/hermes-agent
└─ 2026-06-13/
   ├─ 1301_ARCHITECTURE.md
   ├─ 1301_CODE-SNIPPETS.md
   └─ 1301_QUICK-REFERENCE.md
```

```
├─ 1301_TESTING.md
├─ 1301_API-SURFACE.md
├─ 1314_SESSION-CONTEXT.md
├─ 1314_AGENT-LOOP.md
├─ 1314_SLASH-COMMANDS.md
├─ 1314_PROVIDERS-AUTH.md
└─ 1314_PROFILES-MEMORY-KANBAN.md
```

The Origin Symlink

```
ls -la ~/learn/nousresearch/hermes-agent/origin
# origin → /opt/Code/github.com/nousresearch/hermes-agent
```

`origin/` เป็น symlink ไปหา repo จริง ทำให้ Obsidian link ใน doc สามารถ

`[[origin/gateway/run.py]]` ได้โดยตรง และพอ agent จะ verify ก็แค่

Read `origin/gateway/session.py:617` แทนที่จะต้องจำ path เต็ม

ข้อสังเกต: `~/hermes/hermes-agent` ≠ symlink — เป็น directory จริงที่ติดตั้ง live ส่วน

`~/learn/*/origin` เท่านั้นที่เป็น symlink → repo ที่ clone ไว้ใน `/opt/Code/`

Time-prefix + ชั้น session

ชื่อ doc format คือ `<HHMM>_<TOPIC>.md` — เวลาที่ agent เขียนใน UTC ของวันนั้น ทำให้พอมีหลาย round การ learn ก็ sort ตาม timestamp ได้ทันที:

```
1301_ARCHITECTURE.md    # round 1 — 13:01 UTC
1301_CODE-SNIPPETS.md
...
1314_SESSION-CONTEXT.md # round 2 — 13:14 UTC (เจาะลึกกว่า)
```

พอจะ reference ก็บอกได้ว่า “ดูที่ 1314 round เพราะ round 1 ยังไม่รู้เรื่อง session isolation”

Big files: เข้าไปที่ไหนก่อน

ตารางนี้แสดง entry points ที่มีประโยชน์ที่สุดในแต่ละ file ขนาดใหญ่ — ใช้เป็นจุดเริ่มต้น `grep -n`

หรือ `Read offset:X limit:Y`

หัวข้อ	File	บรรทัด
gateway daemon boot	gateway/run.py	~140
session key build	gateway/session.py	617
slash command dispatch	gateway/run.py	6841
slash handler: /agents	gateway/slash_commands.py	505
slash handler: /goal	gateway/slash_commands.py	1555
defer flow (3s window)	discord/adapter.py	3315
codex import	hermes_cli/auth.py	3676
codex save	hermes_cli/auth.py	3499
login stub (removed)	hermes_cli/auth.py	6430
provider resolution	runtime_provider.py	1254
kanban triage flag	hermes_cli/kanban.py	319
kanban decompose guard	kanban_decompose.py	288
profile describe -text	hermes_cli/main.py	9971

Permalinks สำหรับจุดสำคัญ:

- gateway/run.py:6841 — /agents bypass
- gateway/session.py:617 — session key build
- gateway/slash_commands.py:505 — /agents handler
- gateway/slash_commands.py:1555 — /goal handler
- plugins/platforms/discord/adapter.py:3315 — defer flow start
- hermes_cli/auth.py:3676 — _import_codex_cli_tokens
- hermes_cli/auth.py:3499 — _save_codex_tokens
- hermes_cli/auth.py:6430 — login stub removed
- hermes_cli/runtime_provider.py:1254 — resolve_runtime_provider
- hermes_cli/kanban.py:319 — triage flag
- hermes_cli/kanban_decompose.py:288 — decompose guard
- hermes_cli/main.py:9971 — profile describe --text

วิธีนำทางใน god file

gateway/run.py ใหญ่เกินกว่า Read ทั้ง file — วิธีที่ได้ผล:

```
# 1. grep หา def/class ที่ต้องการ
grep -n \
    "def _handle_agents_command\|class GatewayRunner" \
    ~/.hermes/hermes-agent/gateway/run.py

# 2. Read เฉพาะช่วง offset+limit
# Read gateway/run.py offset:6835 limit:30

# 3. grep cross-file
grep -rn \
    "_handle_agents_command" \
    ~/.hermes/hermes-agent/gateway/
# → definition อยู่ที่ slash_commands.py:505
# dispatch อยู่ที่ run.py:6843
```

อย่า Read run.py ตั้งแต่บรรทัด 1 — ใช้ grep นำทางก่อนเสมอ

Installed vs. ghq source

hermes-agent มีอยู่ 2 ที่ ซึ่งอาจต่าง version กันได้:

Path	ความหมาย
~/.hermes/hermes-agent/	ติดตั้ง live — gateway รันจากที่นี่
/opt/Code/ ... /hermes-agent/	ghq clone — version ที่เราอ่าน/study
ψ/learn/*/origin	symlink → ghq clone

ถ้า hermes update รันแต่ ghq ยังไม่ pull — พอ cite line number ควร verify ว่า grep ที่ตัวไหน

Appendix B: Trap Table — สิ่งที่เจอจริงใน session m5

ทุก trap ในตารางนี้เกิดขึ้นจริงระหว่าง session m5 วันที่ 13 มิ.ย. 2026 — ไม่ใช่ speculation

trap	อาการ	วิธีเลี่ยง
<code>hermes login</code> ถูกถอด	print “removed” แล้ว exit 0	ใช้ <code>hermes auth</code> แทน
<code>codex auth add</code> ต้อง TTY	ค่าที่ y/N → EOFError → default “n”	ใช้ <code>_import_codex_cli_tokens()</code> โดยตรง
<code>profile describe</code> ไม่มี <code>--text</code>	positional arg ผิด	ใส่ <code>--text " ... "</code> เสมอ
<code>kanban decompose</code> fail	<code>status='todo'</code> ผ่าน guard ไม่ได้	สร้างด้วย <code>--trriage</code> flag เสมอ
<code>codex child gave_up</code>	“protocol violation” — spawn ก่อน auth	auth ให้เสร็จก่อน → <code>kanban unblock</code>
<code>slash /agents</code> “did not respond”	auth gate ช้า > 3s Discord timeout	retry; ใช้ @mention แทน
token reset = WS 4004	4004 Authentication Failed → gateway ตาย	<code>pm2 stop</code> → <code>direnv reload</code> → <code>pm2 start</code>
token raw ใน command block	safety hook block command	ใช้ <code>\$(pass show ...)</code> แทน
token raw ใน pm2 env	<code>pm2 env</code> แสดง token leak	ใช้ pass-wrapper script
<code>maw</code> ห้าม raw <code>tmux send-keys</code>	buffer พัง, output ปนกัน	ใช้ <code>maw send-text</code> แทน
<code>maw hey multiline</code> ค้าง	tmux รอ paste ไม่สิ้นสุด	ส่งบรรทัดเดียว หรือส่ง path ของ file
<code>--text</code> กับ <code>--auto</code> exclusive	<code>sys.exit(2)</code> ถ้าใช้พร้อมกัน	เลือกอย่างใดอย่างหนึ่ง
splash cover ถูก crop	<code>object-fit: cover + fixed</code> height	ใช้ <code>width: 100%; height: auto</code>
<code>.envrc</code> stray <code>source /home/nat/..</code>	<code>direnv reload</code> error บน macOS	ลบ line นั้น — migration artifact จาก Linux

รายละเอียด trap สำคัญ (พร้อม permalink):

hermes login ถูกถอด (auth.py:6430–6435): ทำงานได้แต่ print “The ‘hermes login’ command has been removed.” แล้ว exit 0

codex auth add ต้อง TTY (auth.py:6481): เรียก `input()` → EOFError ผ่าน pipe

profile describe error (main.py:9988): check `text_value` and `auto_flag` — ถ้าไม่มี `--text`

flag parser ไม่รู้ว่า string นั้นคือ text

```
kanban decompose guard (kanban_decompose.py:288): check task.status != "trriage" →  
return DecomposeOutcome fail  
slash /agents defer flow (discord/adapter.py:3318–3321): auth gate รันก่อน defer() ถ้า  
gate ซ้ำ > 3s Discord timeout  
profile describe --text ก๊ับ --auto exclusive (main.py:9986–9991):  
if text_value and auto_flag: sys.exit(2)
```

Trap ที่น่าสนใจที่สุด: Codex auth bypass TTY

failure: รัน `hermes auth add openai-codex` จาก subagent (ไม่มี TTY จริง) → ค้างที่ prompt →

EOFError → default `"n"` → auth ไม่เสร็จ → kanban dispatch codex ล้มเหลว

fix: อ่าน `auth.py:6474–6485` แล้วเห็นว่า `_import_codex_cli_tokens()` เป็น pure function อ่าน

`~/.codex/auth.json` โดยตรง และ `_save_codex_tokens()` เขียน `~/.hermes/auth.json` — ทั้งสอง

ไม่ต้อง user input เลย

```
~/.hermes/hermes-agent/venv/bin/python3 -c "  
import sys  
sys.path.insert(0, '/Users/nat/.hermes/hermes-agent')  
from hermes_cli import auth  
# auth.py:3676 - read ~/.codex/auth.json  
t = auth._import_codex_cli_tokens()  
# auth.py:3499 - write ~/.hermes/auth.json  
auth._save_codex_tokens(t)  
"  
  
hermes auth status openai-codex # → logged in
```

บทเรียน: อ่าน source ก่อนสรุปว่า CLI command เป็นวิธีเดียว — function-level bypass มักมีอยู่
แล้ว

ไฟล์ที่ควร read เป็นอันดับแรก ถ้าจะเข้าใจ hermes-agent

1. `ψ/learn/nousresearch/hermes-agent/hermes-agent.md` — index + key insights ของทุก session
learn

2. `ψ/learn/ ... /2026-06-13/1314_SLASH-COMMANDS.md` — เรื่อง defer flow, fingerprint cache, `/agents` real vs. transient เขียนไว้ครบ
 3. `ψ/learn/ ... /2026-06-13/1314_PROVIDERS-AUTH.md` — Codex + Nous auth paths ทั้งหมด
 4. `gateway/run.py:6841` — จุดที่ `/agents` bypass agent-guard และ route ไป handler
- ต้องการ verify claim ใดๆ ให้ grep source ที่ `origin/` (symlink) หรือ `~/.hermes/hermes-agent/` — อย่า trust memory อย่างเดียวเพราะ version drift
-

Hermes Oracle (Claude Sonnet 4.6) — m5 · 13 มิ.ย. 2026 “อ่าน source ก่อน assume — `gateway/run.py` 16K บรรทัด มีคำตอบอยู่ในนั้น”